



BASI DI DATI MULTIMEDIALI

Alessandro Zappatore

Università degli studi di Torino
Anno accademico 2025/2026, 2° semestre

Indice

1	Introduzione	3
1.1	Che cos'è un medium?	3
1.2	Multimedialità	3
1.3	Ipermedialità	3
1.4	I Big Data e i Dati Multimediali	4
1.4.1	Le caratteristiche HMLE dei Big Data Multimediali	4
1.4.2	Ulteriori sfide dei dati multimediali (SIS)	4
1.5	Obiettivi del corso	5
1.6	Sense (Dare un senso ai dati)	5
1.7	Paradigmi di Interazione con i Dati	5
1.7.1	Operazioni di Exploratory Search	6
1.8	Caratteristiche Comuni dei Dati Multimediali	6
1.9	Livelli di Organizzazione dei Dati	7
1.9.1	Basi di Dati Strutturate	7
1.9.2	Basi di Dati Semi-strutturate	7
1.9.3	Basi di Dati Non Strutturate	7
1.10	Features (Caratteristiche estratte)	8
1.11	Gestione delle Imperfezioni	8
1.12	Tipologie di Query	9
1.12.1	Query sui Metadati	9
1.12.2	Query per Esempio (Query-By-Example)	9
1.12.3	Query Semantiche e Basate su Oggetti	9
1.13	QBE: Rappresentazione Visiva e Funzioni di Aggregazione	10
1.13.1	Proprietà fondamentale: La Monotonia	11
1.13.2	Le Funzioni di Aggregazione	11
1.14	Progressività della Ricerca	12
1.15	Come Rappresentare i Dati (Modelli Formali per i Surrogati)	12
1.15.1	1. Insiemi (Sets)	12
1.15.2	2. Vettori (Vectors)	12
1.15.3	3. Stringhe (Strings / Sequences)	12
1.15.4	4. Alberi (Trees)	13
1.15.5	5. Grafi (Graphs)	13
1.16	Tensori	13
1.16.1	Esempi nel contesto Multimediale	14
1.16.2	La sfida computazionale	14

2	Modello Vettoriale (Vector Space Model)	15
2.1	Dimensionalità e Rappresentazione dei Conteggi	15
2.2	Rappresentazione Vettoriale	16
2.2.1	Query come Vettore	16
2.2.2	Misura di Distanza o Similarità	17
2.2.3	Top-k Retrieval	17
2.2.4	Classificazione	17
2.3	Spazio vettoriale	18
2.4	Indipendenza lineare e Base	18
2.5	Prodotto interno e Ortogonalità	19
2.6	Norma e Basi Ortonormali	19
2.6.1	Base Ortonormale	19
2.6.2	Famiglia delle Norme L_p	20
2.7	Modelli Metrici e Distanze	20
2.7.1	Distanza di Minkowski (L_p Metric)	20
2.8	Vantaggi del Modello Metrico	21
2.9	Tipologie di Ricerca nello Spazio Metrico	21
2.10	Altre misure di distanza e similarità	21
2.10.1	Distanza di Mahalanobis	22
2.10.2	Approfondimento: Intuizione sulla Covarianza	22
2.10.3	Esempio Numerico Passo-Passo	23
2.10.4	Distanza Quadratica (Quadratic Form Distance)	24
2.10.5	Intersezione di Istogrammi (Intersection Similarity)	24
2.10.6	Misure basate sugli Angoli (Cosine Similarity)	25
2.10.7	Correlazione di Pearson	26
2.10.8	Divergenza di Kullback-Leibler (KL Divergence)	26
2.10.9	Earth Mover's Distance (EMD)	27
3	Quante features vogliamo?	28
3.1	Dimensionality Curse (Maledizione della Dimensionalità)	28
3.2	Effetto geometrico: Ipersfere e Gusci	28
3.2.1	Power Law	28
3.3	Criteri per la Selezione delle Feature	29
3.3.1	Caratteristiche di una "Buona Feature"	29
3.4	Entropia come misura di qualità	29
3.5	Feature Latenti e Trasformazione dello Spazio	29
3.5.1	Cosa sono le Feature Latenti	30
3.6	Rappresentazione dei testi e Feature Extraction	31
3.6.1	Rappresentazione vettoriale (Vector Space Model)	31
3.6.2	Distribuzione Zipfiana	32
3.6.3	Come definire i pesi vettoriali	32
3.7	Estrazione di Feature dalle Immagini	33
3.7.1	Rappresentazione del Colore	33
3.7.2	Rappresentazioni Lossy e Compressione	35
3.7.3	Istogramma dei colori	37
3.7.4	Istogrammi di colore locali (Spatial Grids)	38
3.7.5	Come scegliere il vocabolario dei colori	38
3.7.6	Confronti tra istogrammi dei colori	38
3.7.7	Texture	39
3.8	Segmentazione e Riconoscimento delle Forme	41
3.9	Rappresentazione e Descrittori di Forma	43
3.10	Rappresentazione delle forme	46
3.10.1	Features locali	47
3.10.2	SIFT (Scale-Invariant Feature Transform)	47

1 Introduzione

1.1 Che cos'è un medium?

Il termine **medium** (plurale media) indica il mezzo o lo strumento attraverso il quale vengono comunicati e trasmessi dei contenuti. La sua natura può variare notevolmente: può trattarsi di testo, immagini, suoni, video, ecc.

È importante notare che, anche all'interno di una singola tipologia di media, possono coesistere diversi **codici comunicativi**. Ad esempio, un'immagine statica non è solo "una foto", ma un insieme complesso di linee, forme, colori e spazi che il nostro cervello decodifica per estrarne un significato.

1.2 Multimedialità

Si parla di **multimedialità** quando avviene la combinazione simultanea di più media per veicolare un messaggio complesso.

- *Esempio*: Un servizio del telegiornale è un prodotto multimediale perché unisce video (immagini in movimento), audio (parlato e suoni ambientali) e spesso testo (sottopancia o titoli).

Dato, Informazione e Conoscenza Quando interagiamo con un media, avviene un processo di elaborazione cognitiva che trasforma il semplice dato in conoscenza.

- **Dato**: L'elemento grezzo percepito (es. vedo un casco con disegnata una tigre).
- **Informazione**: L'elaborazione del dato (es. riconosco che è un casco da sci).
- **Conoscenza**: L'informazione associata al bagaglio culturale pregresso.
 - *Esempio*: Vedo il casco con la tigre (dato), capisco che è attrezzatura da sci (informazione), ma grazie alla mia conoscenza pregressa so che quello è un casco e non una tigre o un pallone data la sua forma tondeggiante.

Soggettività La fruizione di un media è intrinsecamente legata alla **soggettività**. La stessa informazione, veicolata dallo stesso identico media, può essere decodificata in modo diverso da individui diversi.

- *Esempio*: L'immagine di un paesaggio innevato può evocare sensazioni di "bellezza e pace" in un osservatore, e sensazioni di "freddo e desolazione" in un altro.

1.3 Ipermedialità

L'**ipermedialità** riguarda la struttura della navigazione e l'interattività. Un sistema ipermediale prevede che l'informazione non sia fruita in modo lineare (dall'inizio alla fine), ma attraverso **cammini prestabiliti** o scelti dall'utente, tipicamente tramite l'uso di link (collegamenti).

Differenza tra Multimediale e Ipermediale È fondamentale distinguere i due concetti, che possono presentarsi da soli o combinati:

1. **Multimediale ma NON Ipermediale**: Un media che usa diversi codici (audio + video) ma ha una fruizione lineare.
 - *Esempio*: Un film o un video su YouTube (se guardato passivamente). L'utente preme play e il contenuto scorre dall'inizio alla fine senza bivi decisionali.
2. **Ipermediale (o Iperestuale) ma NON Multimediale**: Un contenuto che permette la navigazione non lineare, ma usa un solo tipo di media (solitamente il testo).
 - *Esempio*: Una pagina di Wikipedia composta di solo testo con link blu che rimandano ad altre pagine di solo testo. Questo è tecnicamente un Iperestesto.

3. **Ipermediale E Multimediale:** La combinazione completa. Abbiamo diversi tipi di media (testo, immagini, video) e una struttura di navigazione a link.

- *Esempio:* Un sito web moderno o un videogioco. Posso leggere, guardare video, cliccare su un'immagine per ingrandirla o seguire un link per cambiare argomento.

1.4 I Big Data e i Dati Multimediali

I **Big Data** sono insiemi di dati così grandi e complessi da richiedere tecnologie e metodi analitici avanzati per essere elaborati. Tradizionalmente, vengono definiti attraverso il modello delle "3 V":

- **Volume:** La quantità enorme di dati generati ogni secondo.
- **Velocità:** La rapidità con cui i dati vengono non solo generati, ma anche elaborati e analizzati (spesso in tempo reale).
- **Varietà:** La diversità dei formati e delle tipologie di dati (strutturati come i database tradizionali, o non strutturati come foto, video e post sui social).

L'analisi dei Big Data è fondamentale per individuare tendenze, fare previsioni e supportare le decisioni in settori cruciali come il marketing, la sanità e la finanza. **Queste tre caratteristiche fondanti si applicano perfettamente anche ai dati multimediali.**

1.4.1 Le caratteristiche HMLE dei Big Data Multimediali

Quando i Big Data incontrano il mondo multimediale, assumono tratti specifici che possono essere riassunti nell'acronimo **HMLE**:

- **H - High-dimensional** (Alta dimensionalità): Per descrivere accuratamente un dato multimediale è necessario un numero elevatissimo di variabili. Se per analizzare un testo abbiamo bisogno di uno spazio dimensionale pari all'intero vocabolario di una lingua, per un'immagine in alta risoluzione o un video dobbiamo elaborare milioni di pixel, frequenze audio e frammenti cromatici.
- **M - Multi-modal** (Multimodalità): I dati multimediali combinano diverse "modalità" (o nature) di informazione simultaneamente. Ad esempio, un video di TikTok è multimodale perché contiene immagini in movimento (video), parlato o musica (audio) e spesso sottotitoli o hashtag (testo).
- **L - Inter-Linked** (Interconnessione): I dati non sono isole, ma formano reti complesse. Queste connessioni possono essere:
 - *Esplicite:* Un'amicizia su un social network o un link diretto tra un video e l'articolo di blog che lo incorpora.
 - *Implicite:* Due utenti che non si conoscono ma che hanno guardato per intero lo stesso identico video, creando un legame comportamentale utile per gli algoritmi di raccomandazione.
- **E - Evolving** (In evoluzione): I dati multimediali sono dinamici e cambiano nel tempo. Un testo può essere editato, un video può ricevere nuovi commenti o essere remixato da altri utenti, generando un flusso informativo in continuo mutamento.

1.4.2 Ulteriori sfide dei dati multimediali (SIS)

Oltre al modello HMLE, gestire archivi multimediali enormi comporta l'affrontare tre ulteriori problematiche:

- **S - Sparsity** (Sparsità): All'interno di matrici di dati immense, spesso mancano moltissime informazioni (es. in un catalogo di milioni di film, un singolo utente ne avrà visti solo alcune decine). Questo richiede algoritmi avanzati capaci di gestire gli spazi vuoti e applicare tecniche di imputazione per stimare, nel modo più accurato possibile, il valore mancante o il comportamento probabile dell'utente.

- **I - Imprecision** (Imprecisione): I dati reali contengono spesso "rumore" o difetti intrinseci. Una foto può essere sfocata, un audio disturbato dal vento, o i metadati di un sensore GPS/termico possono avere un certo margine di errore hardware.
- **S - Subjectivity** (Soggettività): Come già visto nell'introduzione, i media sono soggetti all'interpretazione umana. L'analisi automatizzata fatica a catalogare dati soggettivi: un commento testuale può essere letterale o ironico, e un'immagine può trasmettere gioia o tristezza a seconda del contesto culturale dell'osservatore.

1.5 Obiettivi del corso

Il nostro obiettivo è quello di smembrare questi media in modo da poterli memorizzare e successivamente interrogare, per estrarre informazioni utili e significative.

Per esempio se volessimo riconoscere un pomodoro in una foto, dovremmo trovare un modo per distinguere una qualsiasi cosa di colore rosso da un pomodoro. Per fare questo, potremmo cercare di identificare caratteristiche specifiche del pomodoro (forma, dimensione, texture) e confrontarle con quelle presenti nell'immagine.

L'utente finale delle nostre analisi potrebbe essere un essere umano, quindi ci dobbiamo preoccupare della parte di visualizzazione dei risultati.

1.6 Sense (Dare un senso ai dati)

Oggi acquisiamo costantemente dati multimediali e multimodali dall'ambiente circostante. Il nostro compito non è solo memorizzarli ed elaborarli, ma soprattutto assegnare loro un significato nel modo più automatico possibile, riducendo al minimo l'intervento manuale dell'utente. Dobbiamo, in breve, "dare un senso" a questi dati.

Il termine **Sense** racchiude una doppia valenza:

1. **Percezione fisica**: L'atto di acquisire il dato dal mondo reale attraverso dei sensori (es. una fotocamera che cattura la luce, un microfono che registra onde sonore, un sensore tattile).
2. **Comprensione semantica**: L'atto cognitivo o computazionale di attribuire un significato logico all'informazione percepita.

Il Semantic Gap (Divario Semantico) Il nostro obiettivo principale è colmare il **gap semantico**. Si tratta della distanza logica e concettuale che separa il dato grezzo (es. una matrice di pixel colorati) e la sua interpretazione concettuale (es. "questa è l'immagine di un cane che corre in un prato"). Colmare questo divario permette ai sistemi informatici di utilizzare i dati per compiere azioni autonome o aiutare l'essere umano a prendere decisioni informate.

1.7 Paradigmi di Interazione con i Dati

A seconda di quanto l'utente abbia chiari i propri obiettivi e di quali strumenti abbia a disposizione, possiamo distinguere quattro modalità principali di interazione con i dati:

1. Data Processing (Elaborazione dei dati) L'utente sa esattamente *cosa* vuole ottenere e possiede le *funzioni specifiche* per farlo. È un'operazione oggettiva e priva di soggettività.

- *Esempio*: Dato un documento di testo, l'utente applica una funzione per contare quante volte compare la parola "multimediale". Non c'è ambiguità: il sistema esegue un calcolo statistico deterministico.

2. Querying (Interrogazione) L'utente sa *cosa* sta cercando e sa come descriverlo, ma **non ha** la funzione esatta per estrarlo. Delega quindi al sistema il compito di trovare il percorso e il metodo migliore per soddisfare la richiesta.

- *Esempio:* L'utente carica una foto di una scarpa in un motore di ricerca visiva e chiede: "Trova tutte le immagini simili a questa". Il sistema si fa carico di capire quali caratteristiche analizzare per restituire i risultati pertinenti.

3. Navigation (Navigazione) Il livello di astrazione aumenta: l'utente ha un'idea generale di ciò che vuole, ma non sa come descriverlo in modo preciso al sistema, oppure non ha gli strumenti di input adeguati per formulare una query esatta.

- *Esempio:* Un utente vuole comprare dei pantaloni su un e-commerce. Non sa il modello esatto, quindi naviga attraverso il menu a tendina "Abbigliamento → Uomo → Pantaloni", osservando i risultati finché non trova qualcosa che rispecchia la sua necessità implicita.

4. Exploration (Esplorazione) L'utente **non ha un obiettivo preciso**. Si muove all'interno del sistema spinto dalla curiosità o dalla scoperta casuale (serendipity).

- *Esempio:* Aprire TikTok o YouTube e scorrere la home page. L'utente non sta cercando un video specifico, ma esplora i contenuti suggeriti dall'algoritmo fermandosi su quelli che catturano la sua attenzione.

1.7.1 Operazioni di Exploratory Search

Anche l'esplorazione apparentemente casuale segue delle logiche e richiede dei sistemi in grado di supportarla attraverso specifiche operazioni:

- **Analisi:** La ricerca di pattern ricorrenti (tendenze) e di anomalie o eccezioni (outlier) all'interno di grandi moli di dati.
- **Comparazione:** Per navigare tra i dati è necessario che il sistema sappia definire il concetto di "simile" e "diverso", confrontando oggetti complessi attraverso metriche di distanza matematica.
- **Aggregazione:** La capacità di raggruppare (clustering) oggetti simili in categorie logiche per facilitarne la consultazione.
- **Trasformazione:** La modifica della rappresentazione del dato per convenienza computazionale, per estrarre informazioni più salienti o per migliorare le performance degli algoritmi.
 - *Esempio:* Passare dalla codifica colore RGB (Rosso, Verde, Blu) a quella YUV (Luminanza e Crominanza) tramite una trasformazione lineare. Questo non serve necessariamente a comprimere il dato, ma a separare l'informazione sulla luminosità da quella del colore, risultando molto più utile per algoritmi di computer vision o per la compressione video.
- **Visualizzazione:** La traduzione dell'analisi algoritmica (matrici, vettori, grafi) in interfacce grafiche intuitive per l'occhio umano.

1.8 Caratteristiche Comuni dei Dati Multimediali

I dati complessi che cerchiamo di analizzare presentano spesso queste caratteristiche ricorrenti:

- **Multi-dimensionali:** Per descrivere il dato sono necessarie numerose variabili (feature). Ad esempio, un'immagine in scala di grigi richiede una dimensione per ogni singolo pixel per indicarne l'intensità luminosa.
- **Multi-modali:** Contengono nature diverse di informazioni (es. un video contiene contemporaneamente audio, sequenze di immagini e testo sotto forma di sottotitoli).
- **Multi-layer (Multilivello):** Lo stesso ambiente fisico o set di dati può essere descritto attraverso strati sovrapposti.

- *Esempio*: Una mappa cittadina su Google Maps. Il "livello 1" mostra la rete stradale per le automobili, il "livello 2" mostra le piste ciclabili, il "livello 3" la rete dei mezzi pubblici. Sono dati diversi che descrivono lo stesso spazio.
- **Multi-resolution (Multirisoluzione)**: I dati possono essere analizzati a granularità differenti, spaziali o temporali.
 - *Esempio*: I dati del traffico urbano possono essere aggregati e visualizzati ogni 15 minuti, ogni ora o come media settimanale, cambiando completamente la prospettiva dell'analisi.
- **Imprecisi / Ambigui / Soggettivi**: Contengono rumore di fondo (es. un audio fruscante) o dipendono dall'interpretazione umana (es. una recensione sarcastica di un film).

Serie temporali (Time Series): Dati raccolti sequenzialmente nel tempo. Sono *multivariate* se misurano più parametri simultaneamente (es. un macchinario ospedaliero che registra contemporaneamente frequenza cardiaca, pressione sanguigna e saturazione dell'ossigeno in ogni istante t).

1.9 Livelli di Organizzazione dei Dati

1.9.1 Basi di Dati Strutturate

Si basano su una logica rigida, definita a priori tramite uno schema rigido (es. tabelle SQL). Ogni informazione è inserita in campi precisi.

- **Vantaggi**: Dati prevedibili, interrogazioni (query) estremamente veloci e precise, facilità di ottimizzazione.
- **Svantaggi**: Totale incapacità di rappresentare la complessità, la soggettività e la natura non omogenea dei media multimediali.

1.9.2 Basi di Dati Semi-strutturate

Offrono maggiore flessibilità. Lo schema non è rigido ma si "auto-describe" insieme ai dati (es. file JSON, XML o database NoSQL a documenti). Le query si basano sull'attraversamento di cammini o alberi logici.

- **Vantaggi**: Buona organizzazione unita a flessibilità; permette di aggiungere nuovi campi senza rompere la struttura precedente.
- **Svantaggi**: Ancora insufficienti per analizzare il contenuto semantico interno di immagini, video o audio grezzi.

1.9.3 Basi di Dati Non Strutturate

Non esiste nessuno schema predefinito. Il dato è memorizzato nella sua forma grezza. Ne sono un esempio file di testo libero, immagini, audio, video e post sui social network.

Il Concetto di Surrogato Nessun sistema di analisi multimediale lavora mai direttamente confrontando bit per bit il dato originale (sarebbe troppo costoso e inutile). I sistemi lavorano invece su un **Surrogato**. Il surrogato è una rappresentazione formale e compatta che contiene *tutte e sole* le proprietà essenziali del dato originale utili per l'analisi.

- *Esempio*: Se cerco un pomodoro in un database di immagini, il sistema non confronta la foto intera. Estrae dalla foto di input un surrogato (es. un vettore matematico che riassume la forte presenza di rosso e una forma sferica) e lo confronta con i surrogati di tutte le immagini salvate nel database.

1.10 Features (Caratteristiche estratte)

Le **features** sono proprio quegli elementi misurabili che compongono il surrogato. La loro natura cambia radicalmente in base al tipo di media analizzato e all'obiettivo dell'analisi:

- **Nelle Immagini:** Le features classiche includono **colore** (istogrammi), **texture** (ruvidità o pattern visivo) e **forma** (shape). Il rilevamento dei *bordi* (edge detection) è una feature fondamentale per segmentare l'immagine e distinguere dove finisce un pomodoro e inizia lo sfondo, permettendo al sistema di isolare la forma dell'oggetto.
- **Nel Testo:** Frequenza delle parole chiave, analisi sintattica, lunghezza delle frasi, autori, metadati bibliografici.
- **Nei Video (per Sistemi di Raccomandazione):** Invece di analizzare i pixel, se lo scopo è raccomandare un film a un utente (come su Netflix), le features utili saranno i metadati: genere, regista, attori principali, anno di uscita e le valutazioni storiche (rating) lasciate dagli altri utenti.

1.11 Gestione delle Imperfezioni

I sistemi multimediali operano nel mondo reale, il quale è intrinsecamente caotico. Qualsiasi processo che va dall'acquisizione del dato fino alla sua interrogazione è soggetto a diverse forme di imperfezione, errore o approssimazione.

1. Imprecisione nei dati catturati Il dato grezzo può arrivare al sistema già compromesso o rumoroso a causa di limiti fisici o hardware dei sensori.

- *Esempio:* Una fotocamera con la lente sporca, un sensore ottico in condizioni di scarsa luminosità che genera "grana" (rumore digitale), o un microfono esposto al vento. Il sistema deve essere in grado di tollerare e filtrare questo rumore iniziale.

2. Imprecisione nell'estrazione delle feature (Quantizzazione) Quando trasformiamo il dato grezzo in un surrogato (estraendo le feature), siamo costretti a semplificare, introducendo un margine di errore o approssimazione.

- *Esempio (Vocabolario limitato):* Se analizziamo un'immagine a milioni di colori ma decidiamo di estrarre una feature basata su una tavolozza (vocabolario) di soli 16 colori base per risparmiare spazio, il sistema "arrotonderà" i pixel al colore più simile disponibile. Molte sfumature andranno perse.
- *Percezione Macchina vs Umano:* Una macchina può distinguere due codici colore RGB matematicamente diversi che l'occhio umano percepisce come identici. Usare una precisione eccessiva a livello algoritmico potrebbe rivelarsi controproducente, introducendo differenze ("rumore") dove per l'utente umano non esistono.

3. Similarità e Correlazione ingannevole nelle features Le feature scelte potrebbero non essere sufficientemente discriminanti per il dominio applicativo, portando il sistema a confondere oggetti concettualmente diversi ma visivamente simili.

- *Esempio (Pomodoro vs Palla):* Se utilizziamo solo la "forma" (cerchio) e la "dimensione" come feature, il sistema considererà identici un pomodoro e una pallina rossa, fallendo nel recuperarli separatamente.
- *Tolleranza alla deformazione:* Al contrario, a volte desideriamo che il sistema *ignori* alcune imperfezioni. In un sistema di ricerca basato su schizzi a mano libera (Sketch-based Retrieval), l'utente disegna un cerchio imperfetto. Il sistema non deve cercare altre "patate tremolanti", ma deve estrarre la feature astratta "forma circolare" e restituire immagini di cerchi perfetti o oggetti tondi.

4. Imperfezione nella formulazione delle query L'utente spesso fatica a tradurre il proprio intento di ricerca in una query precisa (una manifestazione del *semantic gap*).

- *Esempio*: Nella **Query-By-Example (QBE)**, l'utente fornisce un'immagine di un tramonto sul mare molto arancione per cercare "altri paesaggi marini al tramonto". Il sistema, analizzando i colori, potrebbe restituire immagini di zucche, arance o fiamme (che condividono l'istogramma cromatico), ignorando la composizione (sole + acqua) che l'utente aveva in mente ma non ha saputo specificare.

Il Relevance Feedback (Feedback di Rilevanza): Per mitigare questa incapacità di formulare la query perfetta al primo tentativo, si introduce un ciclo interattivo. Il sistema mostra dei risultati preliminari e l'utente indica quali sono pertinenti ("Mi piace questo", "Non c'entra nulla quest'altro"). Il sistema ricalcola dinamicamente i pesi delle feature (ad esempio, capendo che deve dare meno importanza al colore arancione e più importanza alle linee orizzontali del mare) e affina la ricerca successiva.

1.12 Tipologie di Query

I sistemi informativi multimediali supportano diversi paradigmi per interrogare le basi di dati.

1.12.1 Query sui Metadati

Sono le interrogazioni più tradizionali, basate su testo e logica booleana (Vero/Falso). Lavorano sui dati strutturati (autori, date, tag) associati al media, non sul suo contenuto intrinseco.

- *Esempio*: "Trovami tutti i dipinti di Van Gogh completati nel 1889".
- *Esempio*: "Trovami tutti i film di genere 'Azione' con Brad Pitt".
- *Vantaggio*: Risultati esatti e certi. *Svantaggio*: Richiedono che qualcuno abbia precedentemente e manualmente etichettato a mano tutti i media.

1.12.2 Query per Esempio (Query-By-Example)

L'utente non usa parole, ma fornisce al sistema un oggetto multimediale (un'immagine, uno spezzone audio) chiedendo di trovare elementi correlati. Si dividono in due approcci:

- **Ricerca Esatta (Exact Match)**: Cerca esattamente lo stesso file (es. tramite un controllo crittografico o *hashing*). Utile per trovare copie pirata di un video o immagini duplicate in un archivio.
- **Ricerca per Similarità (Partial/Similarity Match)**: Cerca i record "più vicini" all'esempio fornito nello spazio delle feature. Per funzionare, richiede di definire una metrica di distanza e una soglia di accettabilità (ϵ).
 - *Range Query*: "Restituisci tutte le immagini la cui distanza dall'esempio è inferiore a X ".
 - *k-Nearest Neighbors (k-NN)*: "Restituisci le prime K immagini più simili, indipendentemente da quanto lo siano in termini assoluti".

1.12.3 Query Semantiche e Basate su Oggetti

Sono le query più avanzate, che cercano di comprendere il contenuto profondo della scena scendendo nel dettaglio dei singoli oggetti che la compongono.

- **Similarità Visuale (Low-level)**: Ricerca basata strettamente sull'aspetto grafico grezzo. "Trova immagini con la stessa distribuzione di colori, gli stessi bordi predominanti o la stessa texture di sfondo".
- **Similarità Semantica (High-level)**: Ricerca basata sul significato (concetti). Richiede l'uso di reti neurali o classificatori. "Trova un'immagine che contenga il concetto di 'cane' e il concetto di 'parco'", a prescindere dal fatto che il cane sia bianco, nero, grande o piccolo (le feature visive cambiano, il significato semantico resta).

- **Similarità Spaziale (Topologica):** Non si limita a verificare l'esistenza di oggetti, ma analizza le loro **relazioni spaziali** all'interno della scena (sopra, sotto, a destra, dentro, adiacente).
 - *Esempio:* Una query semantica standard troverebbe un'immagine con una "persona" e un "cane". Una query con vincoli spaziali permette di cercare specificamente "un'immagine in cui una persona si trova *sopra* o *tiene in mano* un cane" (escludendo le foto in cui la persona è sullo sfondo e il cane in primo piano staccato).

1.13 QBE: Rappresentazione Visiva e Funzioni di Aggregazione

Nel paradigma **Query-By-Example (QBE)**, il processo di ricerca non avviene tramite parole chiave, ma attraverso il confronto visivo.

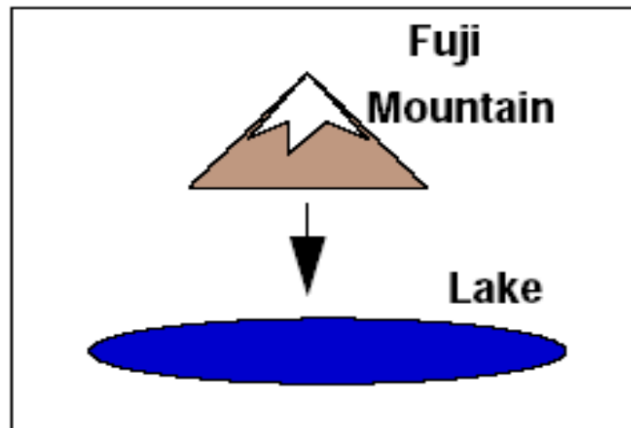


Figura 1: *Input:* L'utente fornisce un'immagine di esempio (es. il Monte Fuji) come query.

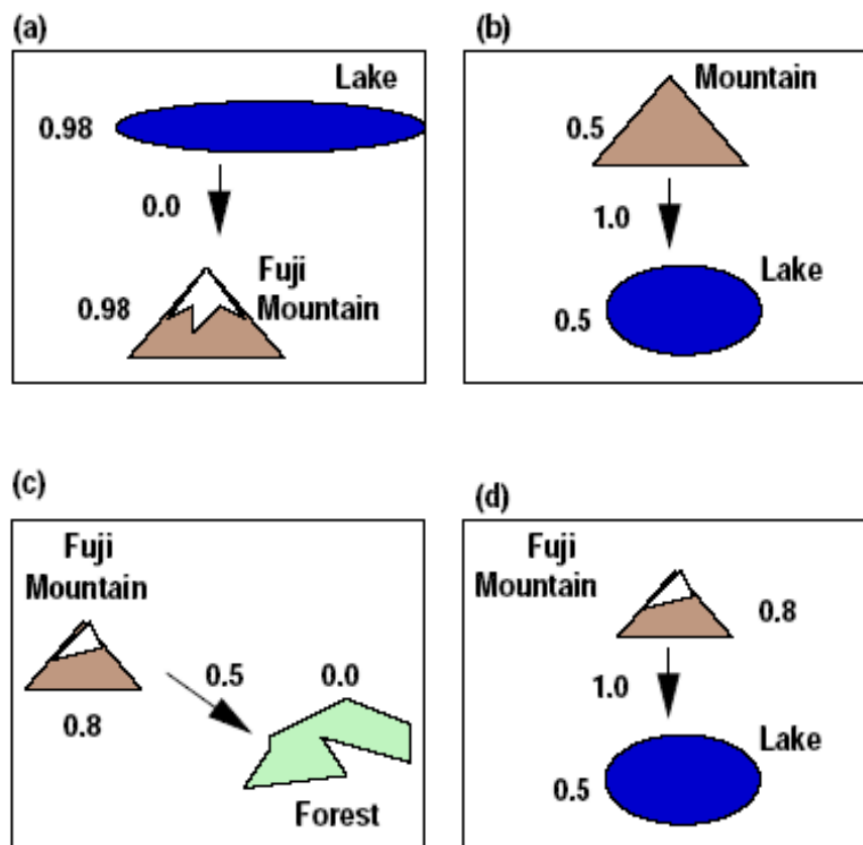


Figura 2: *Output:* Il sistema valuta la similarità visuale e restituisce immagini con caratteristiche (colore, forma) affini.

Quando confrontiamo un'immagine di query con quelle nel database, spesso analizziamo diverse **feature** separatamente (es. una per il colore, una per la texture, una per la forma). Ognuna di queste produce uno "score parziale" di similarità. Dobbiamo poi combinare questi punteggi in un unico **score globale** utilizzando una **funzione di aggregazione** g .

1.13.1 Proprietà fondamentale: La Monotonia

La funzione di aggregazione scelta deve essere **MONOTONA**. Una funzione $g(x_1, \dots, x_m)$ è monotona se, aumentando il valore di uno qualsiasi degli input x_i (migliorando quindi una similarità parziale), il valore del risultato finale non diminuisce mai.

- **Perché è importante?** La monotonia è fondamentale per l'efficienza degli algoritmi di ricerca (come l'algoritmo di Fagin o il Threshold Algorithm). Ci permette di "potare" (pruning) lo spazio di ricerca: se sappiamo che il punteggio parziale massimo possibile per i restanti oggetti non può superare il punteggio del miglior oggetto trovato finora, possiamo fermarci senza scansionare tutto il database.

1.13.2 Le Funzioni di Aggregazione

Esistono diversi modi per combinare gli score parziali, che rispecchiano diverse logiche decisionali:

1. Minimo (Logica AND - Congiunzione) Questa funzione restituisce il valore più basso tra tutti gli score parziali.

$$Score_{globale} = \min(s_{colore}, s_{forma}, s_{texture})$$

- **Comportamento:** Il risultato peggiore "domina" (abbassa) l'intero punteggio.
- **Significato logico:** Corrisponde a una **Congiunzione (AND)**. Richiedo che l'immagine sia simile per colore **E** per forma **E** per texture. Se anche solo una caratteristica è diversa (score basso), l'oggetto viene scartato o penalizzato pesantemente.
- **Esempio:** Cerco una "pallina da tennis gialla". Se trovo un oggetto giallo (score colore alto) ma quadrato (score forma basso), il Minimo prenderà lo score basso della forma. L'oggetto non va bene.

2. Massimo (Logica OR - Disgiunzione) Questa funzione restituisce il valore più alto tra tutti gli score parziali.

$$Score_{globale} = \max(s_{colore}, s_{forma}, s_{texture})$$

- **Comportamento:** Il risultato migliore "domina" (alza) l'intero punteggio.
- **Significato logico:** Corrisponde a una **Disgiunzione (OR)**. Mi basta che l'immagine sia simile per colore **OPPURE** per forma. È una ricerca più lasca.
- **Esempio:** Cerco "qualcosa che ricordi il mare". Potrebbe essere un'immagine blu (colore simile) oppure un'onda disegnata in bianco e nero (forma simile). Basta che una delle due feature sia presente affinché l'oggetto sia considerato rilevante.

3. Media (Compromesso) Calcola la somma dei valori diviso il loro numero, spesso usando una media ponderata per dare più importanza a certe feature rispetto ad altre.

$$Score_{globale} = \frac{1}{n} \sum_{i=1}^n s_i \quad \text{oppure} \quad \sum_{i=1}^n w_i \cdot s_i$$

- **Comportamento:** Tutti i punteggi contribuiscono al risultato finale. Un punteggio molto basso può essere compensato da uno molto alto.
- **Significato:** È la funzione più comune perché offre un bilanciamento tra le varie caratteristiche senza essere drastica come il Minimo o il Massimo.

1.14 Progressività della Ricerca

La **progressività** è una caratteristica fondamentale dei sistemi avanzati di recupero dell'informazione (Information Retrieval) e riguarda l'efficienza computazionale durante l'esplorazione dei risultati. Spesso un utente non vuole vedere tutti i milioni di risultati in una volta sola, ma procede a scaglioni (es. "Mostrami i primi 5 risultati più simili", poi "Mostrami i successivi 5"). Vogliamo che il sistema sia in grado di restituire risultati progressivamente più "lontani" (meno simili all'input) **senza sprecare il lavoro computazionale già svolto**.

- *Il problema*: Se ricalcolassimo da zero le distanze tra la query e l'intero database ogni volta che l'utente chiede la "pagina successiva", il sistema collasserebbe.
- *La soluzione*: Quando il sistema individua il j -esimo elemento più simile, deve mantenere uno stato (ad esempio tramite una coda di priorità o un indice indicizzato). In questo modo, per ottenere l'elemento $(j + 1)$ -esimo, il costo computazionale è minimo, riutilizzando i calcoli precedenti.

1.15 Come Rappresentare i Dati (Modelli Formali per i Surrogati)

Una volta deciso quali feature estrarre da un media, dobbiamo scegliere una struttura matematica e logica per memorizzarle. Questa scelta determinerà quali tipi di query potremo eseguire e con quale efficienza. Di seguito i modelli formali più utilizzati:

1.15.1 1. Insiemi (Sets)

Un insieme è una collezione non ordinata e priva di ripetizioni di elementi. In questo modello, un dato è rappresentato semplicemente dalla presenza o assenza di determinate caratteristiche (logica booleana: 1 o 0).

- **Caratteristiche**: Posso affermare se una proprietà esiste, ma non posso quantificarla.
- *Esempio*: In un'immagine, posso dire che $S = \{\text{rosso}, \text{blu}, \text{albero}\}$, indicando che ci sono questi elementi. Non posso però dire *quanto* rosso c'è o *quanti* alberi ci sono.
- **Uso**: È sufficiente per rispondere a query di tipo esistenziale ("Trova tutte le foto che contengono un gatto"). Tuttavia, è raramente usato da solo per i dati multimediali complessi, poiché perde troppe sfumature essenziali.

1.15.2 2. Vettori (Vectors)

È in assoluto una delle rappresentazioni più utilizzate (il cosiddetto **Vector Space Model**). Un vettore è un array matematico monodimensionale $\mathbf{v} = [v_1, v_2, \dots, v_n]$. La dimensione del vettore (n) è fissa e corrisponde al numero totale di feature che il sistema è in grado di memorizzare. Il vettore supera i limiti degli insiemi perché permette di memorizzare valori continui o discreti per ogni singola feature, catturando due aspetti fondamentali:

- **Conteggio (Frequenza)**: Valori interi che indicano la molteplicità. *Esempio*: Quanti pixel rossi ci sono in un'immagine, o quante volte compare la parola "computer" in un testo.
- **Peso (Importanza)**: Valori continui (spesso normalizzati tra 0 e 1) che indicano la rilevanza di una feature. *Esempio*: Un valore di 0.8 all'indice corrispondente al "rosso" indica che l'immagine ha una forte dominanza di quel colore, mentre 0.1 indica una presenza trascurabile.

1.15.3 3. Stringhe (Strings / Sequences)

Se per descrivere il mio dato multimediale l'**ordine** degli elementi è fondamentale, i vettori e gli insiemi falliscono. Una stringa è una sequenza ordinata di simboli estratti da un alfabeto.

- *Esempio testuale*: Il vettore mi direbbe che le lettere 'A', 'M', 'O', 'R', 'E' sono presenti una volta ciascuna. Ma solo la stringa mi permette di distinguere la parola "AMORE" dalla parola "ROMA" o "ORME".

- *Esempio multimediale*: L'analisi di una sequenza di note musicali (una melodia) o la sequenza temporale dei fotogrammi in un video (dove l'ordine degli eventi cambia completamente la trama).

1.15.4 4. Alberi (Trees)

Quando le feature hanno una natura intrinsecamente **gerarchica** o tassonomica, si utilizza una struttura ad albero.

- *Esempio spaziale*: La scomposizione di un'immagine. La radice dell'albero è l'immagine intera; i figli sono i 4 quadranti in cui viene divisa; i nipoti sono ulteriori sotto-quadranti (come nel modello Quad-tree).
- *Esempio semantico*: Un documento strutturato in Capitoli, Paragrafi e Frasi (es. file XML o JSON).

1.15.5 5. Grafi (Graphs)

Se l'obiettivo principale è rappresentare le **relazioni** e le interconnessioni tra oggetti diversi, il grafo è il modello matematico perfetto. È composto da Nodi (le entità) e Archi (le relazioni tra di esse).

- *Esempio Sociale*: In un social network, ogni utente è un nodo, e un "Mi piace" o un'amicizia è un arco direzionale che li collega.
- *Esempio Visivo (Scene Graph)*: In un'immagine complessa, un nodo è "Persona", un altro nodo è "Cavallo". L'arco che li unisce ha l'etichetta "Cavalca". Questo permette di distinguere un uomo a cavallo da un uomo in piedi accanto a un cavallo.

Esiste una notazione migliore in assoluto? La risposta è **no**. Non esiste un modello universalmente superiore (principio noto in informatica come *No Free Lunch theorem*). Ogni struttura dati è progettata per ottimizzare la risoluzione di un problema specifico:

- Usa i **Vettori** se devi calcolare similarità matematiche veloci (es. riconoscimento volti).
- Usa le **Stringhe** se stai analizzando sequenze temporali (es. riconoscimento vocale).
- Usa i **Grafi** se devi fare inferenza semantica o studiare reti sociali (es. algoritmi di raccomandazione).

Spesso, nei sistemi moderni di Big Data multimediali, questi modelli vengono utilizzati congiuntamente in architetture ibride.

1.16 Tensori

I **tensori** rappresentano la generalizzazione matematica e logica delle strutture dati che abbiamo visto finora. Se pensiamo ai dati come contenitori di numeri organizzati nello spazio, esiste una gerarchia precisa:

- **Scalare (0D)**: Un singolo numero (es. 7).
- **Vettore (1D)**: Una singola riga o colonna di numeri (es. [1, 5, 7]).
- **Matrice (2D)**: Una tabella di numeri con righe e colonne (es. un'immagine in scala di grigi 1024×768).
- **Tensore (n D)**: Una struttura che generalizza il concetto a n dimensioni (o "assi").

Sebbene nell'immaginario comune ci si fermi spesso alla terza dimensione (un cubo di numeri), i tensori possono avere un numero arbitrario di dimensioni ($n > 3$).

1.16.1 Esempi nel contesto Multimediale

Nel mondo dei Big Data e del Deep Learning, i tensori sono la struttura standard per rappresentare l'informazione complessa:

- **Tensore 3D (Immagine a colori):** Un'immagine digitale RGB non è piatta; ha tre livelli di profondità. Viene rappresentata come $Altezza \times Larghezza \times 3$ (Canali Colore).
- **Tensore 4D (Video o Batch):**
 - Un video è una sequenza di immagini nel tempo: $Tempo \times Altezza \times Larghezza \times 3$.
 - Nel training delle reti neurali, si processano gruppi di immagini (batch) contemporaneamente: $NumeroImmagini \times Altezza \times Larghezza \times 3$.

1.16.2 La sfida computazionale

La caratteristica critica dei tensori è il costo di elaborazione. **Le operazioni sui tensori crescono esponenzialmente all'aumentare delle dimensioni.** Se immaginiamo un tensore con D dimensioni, e ogni dimensione ha una grandezza N , il numero totale di elementi da elaborare è N^D .

- *Esempio:* Passare da una matrice 10×10 ($10^2 = 100$ elementi) a un tensore $10 \times 10 \times 10$ ($10^3 = 1000$ elementi) aumenta i dati di un fattore 10 aggiungendo una sola dimensione.

Questo porta spesso alla cosiddetta "**Maledizione della dimensionalità**" (Curse of Dimensionality): all'aumentare delle dimensioni, i dati diventano radi (sparsi) e gli algoritmi di ricerca o di calcolo della distanza (come la Distanza Euclidea) diventano estremamente lenti o meno efficaci nel discriminare la similarità.

2 Modello Vettoriale (Vector Space Model)

Il modello vettoriale è il metodo fondamentale per trasformare dati astratti (immagini, testi, audio) in entità matematiche elaborabili.

2.1 Dimensionalità e Rappresentazione dei Conteggi

Il concetto chiave è che ogni feature misurabile corrisponde a una dimensione geometrica. Il valore della feature (es. un conteggio o un peso) diventa la coordinata su quell'asse.

- **1 Dimensione (Retta):** Se consideriamo una sola feature (es. "quanto è rossa l'immagine"), ogni oggetto viene mappato su una retta.

$$v = [5]$$



Figura 3: Rappresentazione di oggetti su una retta in 1D

- **2 Dimensioni (Piano Cartesiano):** Se consideriamo due feature (es. "quanto è rossa" e "quanto è luminosa"), ogni oggetto è un punto (x, y) su un piano 2D.

$$v = [5, 7]$$

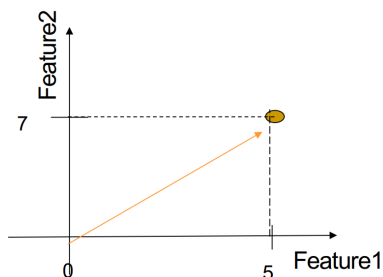


Figura 4: Rappresentazione di oggetti su un piano in 2D

- **3 Dimensioni (Spazio Euclideo):** Con tre feature, l'oggetto diventa un punto (x, y, z) nello spazio 3D.

$$v = [5, 7, 3]$$

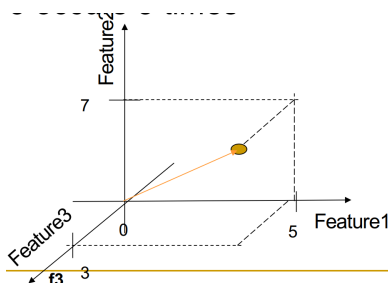


Figura 5: Rappresentazione di oggetti su uno spazio 3D

- **N Dimensioni (Iperspazio):** Generalizzando, se estraiamo N feature, ogni dato multimediale diventa un punto in un iperspazio N -dimensionale. Anche se non possiamo visualizzarlo geometricamente, le regole matematiche rimangono le stesse.

2.2 Rappresentazione Vettoriale

In questo modello, ogni oggetto multimediale O viene rappresentato come un vettore di feature $\mathbf{v}$:

$$\mathbf{v}_O = (f_1, f_2, \dots, f_N)$$

Dove f_i è il valore numerico della i -esima feature.

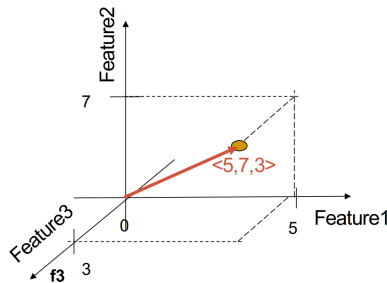


Figura 6: Rappresentazione vettoriale di un oggetto

L'Analogia Spaziale Possiamo immaginare l'intero database come una nuvola di punti in uno spazio multidimensionale (lo *Spazio delle Feature*).

- Immagini visivamente simili avranno vettori con valori simili.
- Di conseguenza, i punti corrispondenti a immagini simili si troveranno vicini nello spazio geometrico.

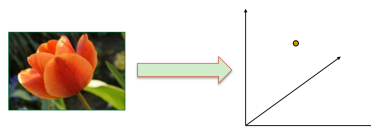


Figura 7: Rappresentazione di un immagine su uno spazio 2D

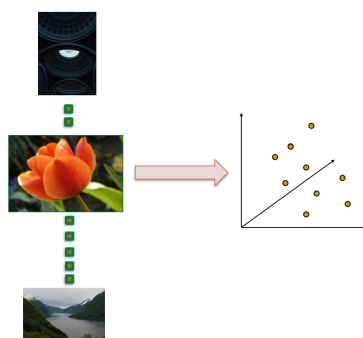


Figura 8: Rappresentazione di più immagini su uno spazio 2D

2.2.1 Query come Vettore

Nel modello vettoriale, la query non è un comando esterno, ma viene tradotta essa stessa in un vettore $\mathbf{q}$ nello stesso spazio dei dati.

- *Esempio*: Se cerco un'immagine simile a una foto di input (Query-By-Example), il sistema estrae le feature dalla foto di input e crea il vettore query $\mathbf{v}_q$.
- L'obiettivo diventa quindi trovare i vettori $\mathbf{v}_d$ (documenti/dati) che sono geometricamente più vicini a $\mathbf{v}_q$.

2.2.2 Misura di Distanza o Similarità

Per quantificare il concetto di "vicinanza" tra la query q e un oggetto d , usiamo delle metriche matematiche. Esiste una relazione inversa tra distanza e similarità:

$$\text{Distanza} \approx \frac{1}{\text{Similarità}}$$

Le metriche più comuni sono:

- **Distanza Euclidea (L_2):** Misura la distanza in linea d'aria tra due punti. È la classica distanza geometrica. Se la distanza è 0, gli oggetti sono identici.

$$d(\mathbf{q}, \mathbf{d}) = \sqrt{\sum_{i=1}^N (q_i - d_i)^2}$$

- **Similarità del Coseno:** Misura l'angolo tra i due vettori, ignorando la loro lunghezza (magnitudine). È molto usata quando conta la proporzione delle feature più che la loro quantità assoluta.

$$\cos(\theta) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|}$$

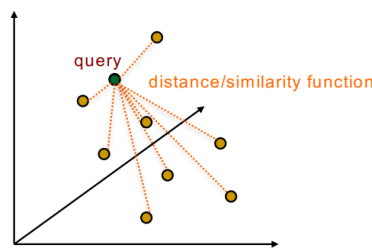


Figura 9: Rappresentazione della distanza tra vettori in uno spazio 2D

2.2.3 Top-k Retrieval

Una volta calcolata la distanza tra il vettore query q e tutti gli altri vettori nel database, il sistema ordina i risultati. Una query Top-k richiede di restituire i k oggetti più vicini alla query. Geometricamente, questo equivale a tracciare una ipersfera centrata nel punto q e allargarla finché non ingloba esattamente k punti.

2.2.4 Classificazione

Il modello vettoriale è la base per la classificazione automatica. Se osserviamo lo spazio delle feature, spesso i dati appartenenti alla stessa categoria (es. "foto di mare" vs "foto di montagna") tendono a raggrupparsi in cluster distinti.

- Un classificatore è un algoritmo che cerca di trovare una superficie di separazione (una linea in 2D, un piano in 3D, un iperpiano in N D) che divida lo spazio in regioni.
- Se un nuovo punto (una nuova immagine) cade nella regione "A", il sistema la classificherà automaticamente come appartenente alla classe "A".

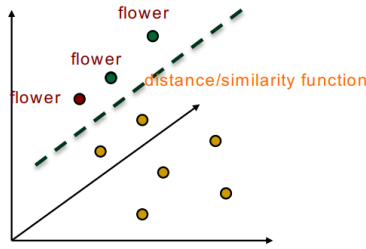


Figura 10: Rappresentazione di oggetti su uno spazio 2D in cui è stata individuata la classe dei fiori

2.3 Spazio vettoriale

Definizione Un insieme $\mathbb{S}$ è definito **spazio vettoriale** su un campo di scalari (come $\mathbb{R}$) se e solo se, per ogni $\vec{v}_i, \vec{v}_j, \vec{v}_k \in \mathbb{S}$ e per ogni scalare $c, d \in \mathbb{R}$, sono soddisfatti i seguenti assiomi:

- **Chiusura e proprietà della somma:**
 - *Commutatività:* $\vec{v}_i + \vec{v}_j = \vec{v}_j + \vec{v}_i$
 - *Associatività:* $\vec{v}_i + (\vec{v}_j + \vec{v}_k) = (\vec{v}_i + \vec{v}_j) + \vec{v}_k$
 - *Elemento neutro:* Esiste un vettore $\vec{0}$ tale che $\vec{v}_i + \vec{0} = \vec{v}_i$
 - *Inverso additivo:* Esiste un vettore $-\vec{v}_i$ tale che $\vec{v}_i + (-\vec{v}_i) = \vec{0}$
- **Chiusura e proprietà del prodotto per scalare:**
 - *Distributività (somma scalari):* $(c + d) \cdot \vec{v}_i = c \cdot \vec{v}_i + d \cdot \vec{v}_i$
 - *Distributività (somma vettori):* $c \cdot (\vec{v}_i + \vec{v}_j) = c \cdot \vec{v}_i + c \cdot \vec{v}_j$
 - *Compatibilità prodotto:* $(cd) \cdot \vec{v}_i = c \cdot (d\vec{v}_i)$
 - *Identità moltiplicativa:* $1 \cdot \vec{v}_i = \vec{v}_i$

Gli elementi di $\mathbb{S}$ sono chiamati **vettori**.

2.4 Indipendenza lineare e Base

Definizione di Indipendenza Sia $V = \{\vec{v}_1, \vec{v}_2, \dots, \vec{v}_n\}$ un insieme di vettori in uno spazio $\mathbb{S}$. Si dice che i vettori sono **linearmente indipendenti** se l'unica combinazione lineare che produce il vettore nullo è quella in cui tutti i coefficienti sono nulli:

$$\sum_{i=1}^n c_i \cdot \vec{v}_i = \vec{0} \quad \Rightarrow \quad c_1 = c_2 = \dots = c_n = 0$$

Se questa condizione non è verificata, significa che almeno un vettore è "ridondante" (può essere espresso come somma degli altri).

Definizione di Base Un insieme V è detto **base** di $\mathbb{S}$ se soddisfa due condizioni:

1. **Indipendenza Lineare:** I vettori non sono ridondanti.
2. **Completezza (Generazione):** Ogni vettore $\vec{u} \in \mathbb{S}$ può essere generato come combinazione lineare dei vettori della base:

$$\vec{u} = \sum_{i=1}^n c_i \cdot \vec{v}_i$$

Il numero di vettori che compongono la base definisce la **dimensione** dello spazio vettoriale.

Esempio Pratico Se descriviamo un oggetto usando 3 feature, stiamo operando in uno spazio a 3 dimensioni. Avere una quarta feature che è una semplice combinazione matematica delle prime tre (es. $f_4 = f_1 + f_2$) non aggiunge informazione, ma solo ridondanza. Una base efficiente conterrebbe solo le prime 3 feature.

2.5 Prodotto interno e Ortogonalità

Definizione Un **prodotto interno** (o scalare) su $\mathbb{S}$ è una funzione $\mathbb{S} \times \mathbb{S} \rightarrow \mathbb{R}$ che associa a due vettori un numero reale, soddisfacendo:

- **Commutatività:** $\vec{u} \cdot \vec{v} = \vec{v} \cdot \vec{u}$
- **Distributività:** $(c_1 \vec{u} + c_2 \vec{v}) \cdot \vec{w} = c_1(\vec{u} \cdot \vec{w}) + c_2(\vec{v} \cdot \vec{w})$
- **Positività:** $\vec{u} \cdot \vec{u} > 0$ se $\vec{u} \neq \vec{0}$

Ortogonalità Due vettori sono **ortogonali** se il loro prodotto interno è nullo:

$$\vec{u} \cdot \vec{v} = 0$$

In termini geometrici, significa che formano un angolo di 90° . Un insieme di vettori mutuamente ortogonali è sempre linearmente indipendente.

Interpretazione Geometrica Il prodotto interno è legato alla proiezione di un vettore sull'altro:

$$\vec{u} \cdot \vec{v} = \|\vec{u}\| \|\vec{v}\| \cos(\theta)$$

Dove θ è l'angolo tra i due vettori. Se i vettori sono normalizzati (lunghezza 1), il prodotto scalare corrisponde esattamente al coseno dell'angolo (Cosine Similarity).

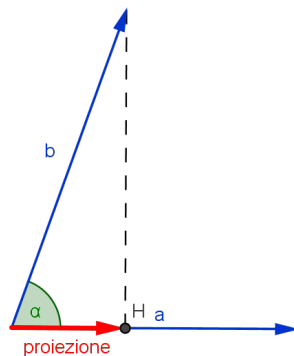


Figura 11: Il prodotto interno rappresenta la proiezione di un vettore sull'altro

2.6 Norma e Basi Ortonormali

Definizione La **norma** $\|\vec{v}\|$ è una funzione che assegna una lunghezza (magnitudine) strettamente positiva a un vettore.

2.6.1 Base Ortonormale

Una base $V = \{\vec{v}_1, \dots, \vec{v}_n\}$ è **ortonormale** se valgono due condizioni:

1. I vettori sono a due a due ortogonali (prodotto scalare = 0).
2. I vettori sono normalizzati (prodotto scalare con se stessi = 1).

Formalmente, usando il delta di Kronecker $\delta_{i,j}$:

$$\vec{v}_i \cdot \vec{v}_j = \delta_{i,j} = \begin{cases} 1 & \text{se } i = j \\ 0 & \text{se } i \neq j \end{cases}$$

2.6.2 Famiglia delle Norme L_p

La definizione generalizzata di lunghezza è data dalla norma L_p . Dato un vettore $\vec{x} = (x_1, \dots, x_n)$:

$$\|\vec{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$$

Casi notevoli:

- $p = 1$ (**Norma Manhattan**): Somma dei valori assoluti.
- $p = 2$ (**Norma Euclidea**): La classica lunghezza geometrica "in linea d'aria".
- $p = \infty$ (**Norma di Chebyshev**): Determina la lunghezza basandosi solo sulla componente più grande del vettore:

$$\|\vec{x}\|_\infty = \max_i |x_i|$$

2.7 Modelli Metrici e Distanze

Per confrontare due oggetti (surrogati) $\vec{x}$ e $\vec{y}$, definiamo una funzione di **distanza** $d(\vec{x}, \vec{y})$.

Una **metrica** è una regola matematica che assegna un numero alla distanza tra due oggetti in modo coerente. Quel numero deve essere interpretabile come una vera nozione di "quanto sono diversi" due punti nello spazio delle feature. In pratica, la metrica è ciò che trasforma la similarità in un problema geometrico: più il valore è piccolo, più gli oggetti sono simili; più è grande, più sono diversi.

Questa proprietà è fondamentale perché algoritmi come *range search*, *k-NN*, clustering e classificazione si basano proprio sull'ordinamento dei punti in base alle distanze. Se la funzione di distanza non è una metrica, il concetto di "vicinanza" può diventare ambiguo e i risultati del retrieval possono essere instabili o poco affidabili.

Una funzione è una metrica valida se soddisfa gli assiomi:

1. **Non negatività**: $d(\vec{x}, \vec{y}) \geq 0$ e $d(\vec{x}, \vec{x}) = 0$.
2. **Simmetria**: $d(\vec{x}, \vec{y}) = d(\vec{y}, \vec{x})$.
3. **Disuguaglianza triangolare**: $d(\vec{x}, \vec{y}) \leq d(\vec{x}, \vec{z}) + d(\vec{z}, \vec{y})$.

2.7.1 Distanza di Minkowski (L_p Metric)

È la generalizzazione della distanza tra due punti A e B in uno spazio N -dimensionale:

$$d_p(A, B) = \left(\sum_{i=1}^N |a_i - b_i|^p \right)^{1/p}$$

1. Distanza Manhattan (L_1 Metric) Si ottiene ponendo $p = 1$. Corrisponde alla distanza percorsa muovendosi solo lungo gli assi (come un taxi in una città a griglia).

$$d_1(A, B) = \sum_{i=1}^N |a_i - b_i|$$

Utilizzo: Spesso preferita nella Computer Vision per confrontare istogrammi di colore, in quanto cattura meglio la similarità percepita dall'occhio umano ed è robusta agli outlier.

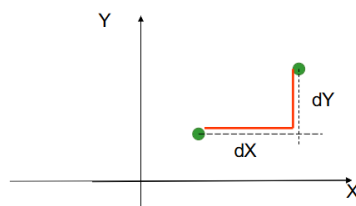


Figura 12: Distanza L1 (Manhattan): somma dei cateti

2. Distanza Euclidea (L_2 Metric) Si ottiene ponendo $p = 2$. È la distanza più comune.

$$d_2(A, B) = \sqrt{\sum_{i=1}^N (a_i - b_i)^2}$$

Utilizzo: Standard per molti algoritmi di clustering, ma soffre in spazi ad altissima dimensionalità.

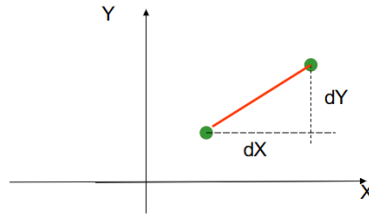


Figura 13: Distanza L2 (Euclidea): ipotenusa

3. Distanza di Chebyshev (L_∞ Metric) Si ottiene con $p \rightarrow \infty$. La distanza è determinata unicamente dalla singola dimensione in cui i due oggetti differiscono maggiormente.

$$d_\infty(A, B) = \max_i |a_i - b_i|$$

2.8 Vantaggi del Modello Metrico

Il modello metrico offre diversi vantaggi pratici nella valutazione della similarità tra immagini e oggetti multimediali:

- È particolarmente adatto per alcune forme di valutazione della similarità, ad esempio i confronti basati sul colore.
- È coerente con approcci ampiamente utilizzati nelle comunità di *computer vision* e *pattern recognition*.
- I risultati sperimentali suggeriscono che la metrica L_1 può catturare meglio la nozione umana di similarità tra immagini.
- Rende relativamente semplice l'indicizzazione dei dati, modellati come vettori di proprietà, mediante tecniche classiche di indicizzazione multidimensionale.

2.9 Tipologie di Ricerca nello Spazio Metrico

Una volta definita la metrica, possiamo eseguire interrogazioni sul database:

- **Range Search (Ricerca per raggio):** "Trova tutti i vettori che distano meno di r dalla query". Geometricamente, si selezionano i punti dentro un'ipersfera di raggio r .
- **k-Nearest Neighbors (k-NN o Top-k):** "Trova i k vettori più vicini alla query". Il raggio di ricerca non è fisso, ma si espande finché non ingloba k elementi.

2.10 Altre misure di distanza e similarità

Oltre alle metriche L_p standard, esistono misure più sofisticate progettate per gestire correlazioni tra variabili, distribuzioni di probabilità o invarianze di scala.

2.10.1 Distanza di Mahalanobis

La Distanza di Mahalanobis è una misura di distanza tra un punto e una distribuzione (o tra due punti in uno spazio con assi correlati). A differenza della distanza Euclidea, che tratta tutte le dimensioni allo stesso modo e come indipendenti, la Mahalanobis tiene conto della correlazione tra le variabili e della varianza di ciascuna. Geometricamente, mentre la distanza Euclidea forma "sfere" intorno al centro, la distanza di Mahalanobis forma "ellissoidi" orientati secondo la distribuzione dei dati.

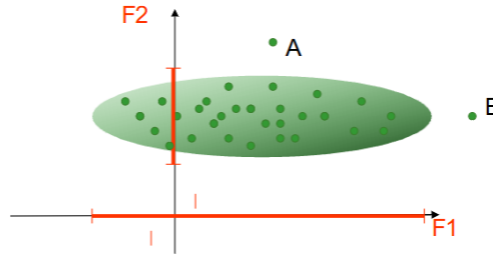


Figura 14: Distanza di Mahalanobis

Il ruolo della Covarianza Se la direzione di massima variabilità dei dati non è allineata con gli assi cartesiani (es. c'è una correlazione tra le feature x e y), la distanza Euclidea può essere ingannevole. La Mahalanobis "schiaccia" o "dila" lo spazio in base alla matrice di covarianza S , trasformando l'ellissoide di distribuzione in una sfera unitaria.

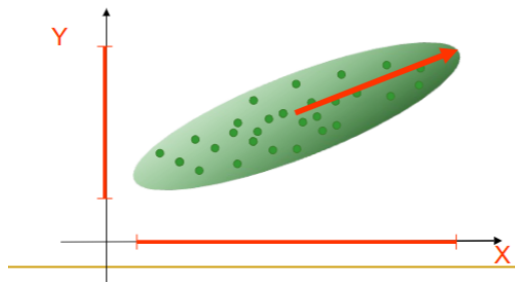


Figura 15: Distanza di Mahalanobis in presenza di feature non allineate con gli assi

Definizioni Formali Siano $\vec{a}$ e $\vec{b}$ due vettori (o un vettore e la media della distribuzione).

- **Matrice di Covarianza (S o Σ):** Una matrice simmetrica che descrive come le feature variano insieme. L'elemento (i, j) è la covarianza tra la feature i e la feature j .

$$S_{xy} = Cov(x, y) = E((x - \mu_x)(y - \mu_y))$$

- **Formula della Distanza:**

$$\Delta_{Mah}(\vec{a}, \vec{b}) = \sqrt{(\vec{a} - \vec{b})^T S^{-1} (\vec{a} - \vec{b})}$$

L'uso dell'inversa della matrice di covarianza (S^{-1}) serve a "normalizzare" i dati, rimuovendo l'effetto delle correlazioni e delle diverse scale. Se le variabili sono incorrelate e hanno varianza unitaria (S è la matrice identità), la distanza di Mahalanobis si riduce alla distanza Euclidea.

2.10.2 Approfondimento: Intuizione sulla Covarianza

Per comprendere la distanza di Mahalanobis, è essenziale capire cosa rappresenta la matrice di covarianza S (o Σ). Essa non ci dice solo quanto i dati sono "sparsi" (varianza), ma anche come si muovono insieme.

Immaginiamo di avere due feature, x (Altezza) e y (Peso):

- **Varianza (σ^2):** È la diagonale della matrice. Ci dice quanto variano i dati lungo un singolo asse. Se la varianza di x è alta e quella di y è bassa, la nuvola di punti sarà allungata orizzontalmente.

- **Covarianza (σ_{xy}):** Sono gli elementi fuori dalla diagonale. Ci dicono se esiste una relazione lineare:
 - **Positiva:** Se x cresce, anche y tende a crescere (es. altezza e peso). La nuvola di punti è inclinata verso destra (/).
 - **Negativa:** Se x cresce, y tende a diminuire. La nuvola è inclinata verso sinistra (\).
 - **Zero:** Le variabili sono indipendenti. La nuvola è circolare o un'ellisse allineata perfettamente agli assi (nessuna inclinazione).

Perché invertiamo la matrice (S^{-1})? Nella formula usiamo l'inversa della covarianza. Intuitivamente, dividere per la varianza significa penalizzare le deviazioni lungo gli assi dove c'è poca variabilità (dove siamo molto sicuri del valore medio) e perdonare le deviazioni lungo gli assi con alta variabilità (dove è normale trovare dati sparsi).

2.10.3 Esempio Numerico Passo-Passo

Immaginiamo uno scenario in cui la distanza Euclidea fallisce nel giudicare la similarità, mentre la Mahalanobis ci dà la risposta corretta.

Sia data una distribuzione di punti con media nell'origine $\vec{\mu} = [0, 0]^T$. Le feature x e y sono positivamente correlate (es. $y \approx x$). Definiamo la matrice di covarianza S :

$$S = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Consideriamo due punti da analizzare, A e B :

- Punto $A = [1, 1]^T$ (allineato con la correlazione).
- Punto $B = [1, -1]^T$ (opposto alla correlazione).

1. Calcolo della Distanza Euclidea (L_2) Calcoliamo la distanza geometrica semplice dall'origine $(0, 0)$.

$$\Delta_{Euc}(A, \mu) = \sqrt{(1-0)^2 + (1-0)^2} = \sqrt{2} \approx 1.41$$

$$\Delta_{Euc}(B, \mu) = \sqrt{(1-0)^2 + (-1-0)^2} = \sqrt{2} \approx 1.41$$

Risultato Euclideo: I due punti sono considerati **identici** in termini di distanza. Il sistema non riesce a distinguere che A è "normale" (segue il trend) e B è un'anomalia (va contro il trend).

2. Calcolo della Distanza di Mahalanobis Per prima cosa, dobbiamo calcolare l'inversa della matrice di covarianza S^{-1} . Il determinante è $|S| = (2 \cdot 2) - (1 \cdot 1) = 3$.

$$S^{-1} = \frac{1}{|S|} \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} = \begin{bmatrix} 0.66 & -0.33 \\ -0.33 & 0.66 \end{bmatrix}$$

Applichiamo la formula $\Delta_{Mah} = \sqrt{\vec{x}^T S^{-1} \vec{x}}$ per i due punti.

Per il punto A (1, 1):

$$\begin{aligned} \Delta_{Mah}^2(A) &= [1 \quad 1] \cdot \frac{1}{3} \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\ &= \frac{1}{3} [1 \quad 1] \cdot \begin{bmatrix} (2-1) \\ (-1+2) \end{bmatrix} = \frac{1}{3} [1 \quad 1] \cdot \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{1}{3}(1+1) = 0.66 \\ \Delta_{Mah}(A) &\approx 0.81 \end{aligned}$$

Per il punto B (1, -1):

$$\begin{aligned} \Delta_{Mah}^2(B) &= [1 \quad -1] \cdot \frac{1}{3} \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ -1 \end{bmatrix} \\ &= \frac{1}{3} [1 \quad -1] \cdot \begin{bmatrix} (2+1) \\ (-1-2) \end{bmatrix} = \frac{1}{3} [1 \quad -1] \cdot \begin{bmatrix} 3 \\ -3 \end{bmatrix} = \frac{1}{3}(3+3) = 2 \\ \Delta_{Mah}(B) &\approx 1.41 \end{aligned}$$

Conclusione dell'esempio Nonostante geometricamente siano alla stessa distanza dal centro, la distanza di Mahalanobis ci dice che:

- Il punto A (distanza 0.81) è molto **più vicino** statisticamente, perché si trova lungo la direzione principale dei dati.
- Il punto B (distanza 1.41) è molto **più lontano**, perché si trova in una zona "rara" o anomala rispetto alla distribuzione della covarianza.

2.10.4 Distanza Quadratica (Quadratic Form Distance)

Questa distanza è fondamentale quando le dimensioni del vettore feature non sono indipendenti, ma esiste una somiglianza intrinseca tra di esse (fenomeno noto come "Crosstalk" o correlazione tra bin).

- *Esempio:* Nel confronto di istogrammi di colore. Nella distanza Euclidea standard, un pixel "Rosso" è equidistante da un pixel "Arancione" e da un pixel "Blu". Percettivamente, però, il rosso è molto più simile all'arancione. Un errore di classificazione tra rosso e arancione dovrebbe pesare meno di un errore tra rosso e blu.

Per modellare questo comportamento, definiamo prima una **matrice di similarità** A , dove ogni elemento a_{ij} rappresenta quanto la feature i è simile alla feature j (valore alto = molto simile). La formula della distanza quadratica utilizza l'**inversa** di questa matrice (A^{-1}):

$$\Delta_{quad}(\vec{x}, \vec{y}) = \sqrt{(\vec{x} - \vec{y})^T A^{-1} (\vec{x} - \vec{y})}$$

Interpretazione dell'Inversa (A^{-1}) L'utilizzo della matrice inversa è cruciale per la corretta ponderazione degli errori:

1. **Similarità e Pesì:** Se due feature i e j sono molto simili (es. Rosso e Arancione), il valore corrispondente nella matrice di similarità A_{ij} sarà **alto**.
2. **Inversione:** Passando alla matrice inversa A^{-1} , la relazione si inverte. Intuitivamente, a valori alti in A corrispondono valori ridotti (o pesi minori) nella struttura di A^{-1} che agisce sulle differenze incrociate.
3. **Effetto sulla Distanza:** Quando calcoliamo la distanza, la differenza tra i valori delle feature ($\vec{x} - \vec{y}$) viene moltiplicata per questi pesi contenuti in A^{-1} .

In sintesi:

- **Feature simili** (A_{ij} alto) $\rightarrow$ Peso nell'inversa (A^{-1}) basso $\rightarrow$ La differenza impatta **poco** sulla distanza totale.
- **Feature diverse** (A_{ij} basso o nullo) $\rightarrow$ Peso nell'inversa più rilevante $\rightarrow$ La differenza impatta **molto** sulla distanza totale.

2.10.5 Intersezione di Istogrammi (Intersection Similarity)

È una misura di similarità molto usata in Computer Vision per il riconoscimento di oggetti basato sul colore. Calcola l'area di sovrapposizione tra due istogrammi.

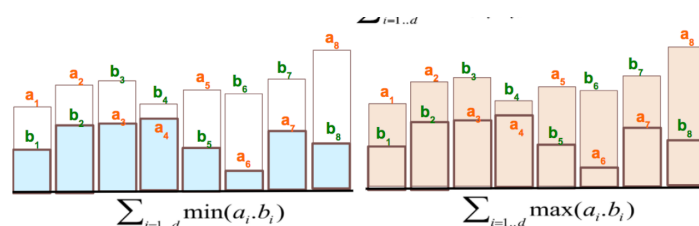


Figura 16: Intersezioni tra due istogrammi

$$sim_{int}(\vec{a}, \vec{b}) = \frac{\sum_{i=1}^n \min(a_i, b_i)}{\sum_{i=1}^n \max(a_i, b_i)}$$

Il risultato varia da 0 (nessuna sovrapposizione) a 1 (istogrammi identici). È robusta a variazioni di sfondo e occlusioni parziali.

2.10.6 Misure basate sugli Angoli (Cosine Similarity)

Queste misure valutano l'orientamento dei vettori nello spazio, ignorando (o normalizzando) la loro magnitudine (lunghezza). Sono particolarmente utili quando la "quantità" assoluta non conta quanto la "proporzione".

- *Esempio Immagini*: Due foto identiche, ma una più luminosa dell'altra, avranno vettori di colore con lunghezze diverse ma stessa direzione.
- *Esempio Testo*: Un documento breve e uno lungo che trattano lo stesso argomento avranno frequenze di parole diverse, ma distribuzione proporzionale simile.

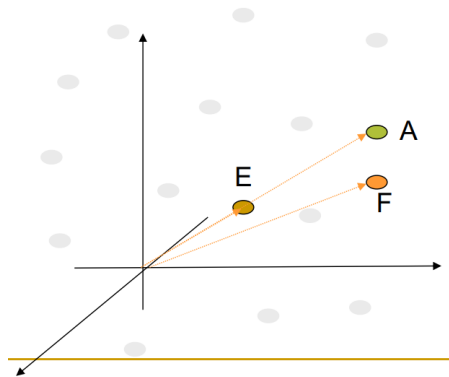


Figura 17: 2 vettori con stessa direzione ma lunghezza diversa

In questo esempio, i vettori A ed F sono più vicini quindi più simili tra loro, ma il vettore E e il vettore A hanno la stessa direzione, quindi stesso angolo rispetto all'origine, quindi sono più simili tra loro rispetto al vettore F per altri fattori.

Dati due vettori $\vec{a}$ e $\vec{b}$:

1. Dot Product (Prodotto Scalare)

$$sim_{dot}(\vec{a}, \vec{b}) = \vec{a} \cdot \vec{b} = \sum_{i=1}^n a_i b_i$$

Il valore dipende dalla lunghezza dei vettori, quindi non è limitato.

2. Cosine Similarity Normalizza il prodotto scalare per le lunghezze dei vettori, ottenendo il coseno dell'angolo θ tra essi:

$$sim_{cos}(\vec{a}, \vec{b}) = \cos(\theta) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$$

- 1: Vettori allineati (massima similarità).
- 0: Vettori ortogonali (non correlati).
- -1: Vettori opposti.

2.10.7 Correlazione di Pearson

La correlazione di Pearson misura la relazione lineare tra due variabili. Nel contesto dei vettori, può essere vista come una Cosine Similarity applicata ai vettori "centrati" (cioè sottraendo la media di ciascun vettore dai suoi componenti).

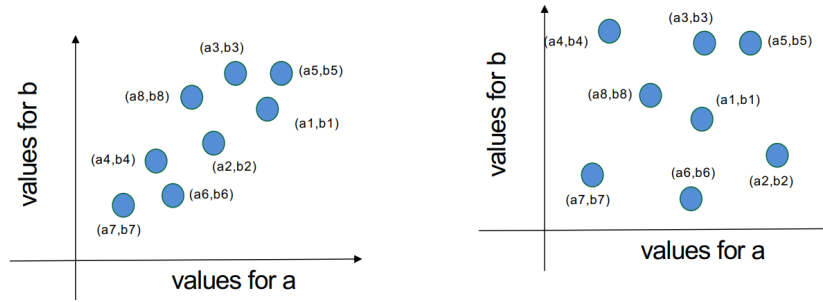


Figura 18: 2 esempi di correlazione di Pearson: a sinistra correlazione positiva, a destra nessuna correlazione

$$\rho(\vec{a}, \vec{b}) = \frac{\sum (a_i - \bar{a})(b_i - \bar{b})}{\sqrt{\sum (a_i - \bar{a})^2} \sqrt{\sum (b_i - \bar{b})^2}}$$

È utile per capire se al variare delle feature di un oggetto, variano linearmente anche quelle dell'altro, indipendentemente dai valori assoluti (shift e scale invariant).

2.10.8 Divergenza di Kullback-Leibler (KL Divergence)

La KL Divergence non è propriamente una distanza (non è simmetrica: $\Delta_{KL}(a, b) \neq \Delta_{KL}(b, a)$), ma misura la divergenza tra due distribuzioni di probabilità P e Q . È spesso usata quando i vettori rappresentano istogrammi normalizzati ($\sum p_i = 1$).

$$\Delta_{KL}(\vec{a}, \vec{b}) = \sum_i a_i \log \left(\frac{a_i}{b_i} \right)$$

Interpretazione della Teoria dell'Informazione L'Entropia $H(P)$ misura l'incertezza o la quantità media di informazione necessaria per descrivere eventi di una distribuzione P :

$$H(P) = - \sum p_i \log_2(p_i)$$

La KL Divergence misura la "sorpresa" o l'inefficienza (bit extra necessari) quando si usa la distribuzione Q per approssimare la vera distribuzione P . Se $\Delta_{KL}(\vec{a}, \vec{b}) = 0$, le distribuzioni sono identiche. Più alto è il valore, più informazione si perde usando b per descrivere a .

Lo spazio dedicato alla singola codifica è:

$$sp(e_1) = \log_2 \frac{1}{prob}$$

Quando devo comprimere un documento costituito da N termini, se gli eventi fossero equiprobabili potrei usare il logaritmo, invece se non lo sono, avrò che per ciascuno $\log \frac{1}{p_i}$, per comprimere l'intero documento mi serviranno $\sum_{i=1}^N p_i \log \frac{1}{p_i}$.

Nella kl distance uso $\log_2 \frac{1}{q_i}$ mi chiedo quanto sarà l'errore di codifica cambiando distribuzione di probabilità e lo ottengo

$$\sum_{i=1}^N p_i \log_2 \frac{1}{q_i} - \sum_{i=1}^N p_i \log_2 \frac{1}{p_i} = \sum_{i=1}^N p_i \log_2 \frac{p_i}{q_i}$$

Così facendo ottengo la formula della kl divergence.

Se voglio renderla simmetrica, posso fare la media tra le due versioni.

2.10.9 Earth Mover's Distance (EMD)

L'EMD (nota anche come metrica di Wasserstein per distribuzioni discrete) è una misura robusta per confrontare due distribuzioni di probabilità o istogrammi. A differenza delle distanze bin-to-bin (come la Minkowski) che confrontano solo indici corrispondenti, l'EMD tiene conto della "distanza di terra" (*ground distance*) tra i bin, permettendo di gestire shift e deformazioni.

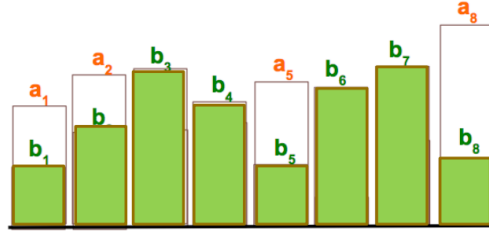


Figura 19: Esempio di Earth Mover's Distance: spostamento di massa tra due istogrammi

L'Analogia del Trasporto Immaginiamo due istogrammi come due accumuli di terra:

- Il vettore $\vec{a} = \langle a_1, \dots, a_n \rangle$ rappresenta la configurazione iniziale (le colline di terra).
- Il vettore $\vec{b} = \langle b_1, \dots, b_n \rangle$ rappresenta la configurazione target (le buche da riempire).

L'EMD calcola il **costo minimo** di lavoro necessario per trasformare $\vec{a}$ in $\vec{b}$ spostando la "massa" di probabilità.

Definizione Formale del Flusso Definiamo ab_{ij} come la quantità di flusso (probabilità) spostata dal bin i di $\vec{a}$ al bin j di $\vec{b}$. Ogni spostamento ha un costo unitario c_{ij} , che rappresenta la distanza semantica o fisica tra la feature i e la feature j . Il problema viene modellato come un problema di *Trasporto Ottimo* (Linear Programming):

1. Minimizzare il Lavoro Totale:

$$W = \sum_{i=1}^n \sum_{j=1}^n c_{ij} \cdot ab_{ij}$$

2. Soggetto ai vincoli:

- **Non negatività:** $ab_{ij} \geq 0$ (non posso spostare massa negativa).
- **Vincolo sulla Sorgente:** La somma della massa che esce da a_i deve eguagliare il valore originale a_i :

$$\sum_{j=1}^n ab_{ij} = a_i$$

- **Vincolo sulla Destinazione:** La somma della massa che arriva in b_j deve eguagliare il valore target b_j :

$$\sum_{i=1}^n ab_{ij} = b_j$$

- **Conservazione Totale:** La massa totale spostata è normalizzata (solitamente a 1 per distribuzioni di probabilità):

$$\sum_{i=1}^n \sum_{j=1}^n ab_{ij} = 1$$

La distanza finale è il lavoro normalizzato per il flusso totale:

$$\Delta_{EMD}(\vec{a}, \vec{b}) = \frac{\sum_{i=1}^n \sum_{j=1}^n c_{ij} \cdot ab_{ij}}{\sum_{i=1}^n \sum_{j=1}^n ab_{ij}} = \sum_{i=1}^n \sum_{j=1}^n cost_{ij}$$

Se i vettori sono normalizzati (somma = 1), il denominatore è 1 e la formula si semplifica nella somma dei costi ponderati.

3 Quante features vogliamo?

La scelta del numero di dimensioni N è un compromesso critico. Vorremmo il minor numero possibile di features per efficienza, ma un numero sufficiente per non perdere potere descrittivo.

3.1 Dimensionality Curse (Maledizione della Dimensionalità)

Al crescere del numero di dimensioni N , si verificano fenomeni controintuitivi che degradano le prestazioni dei sistemi di retrieval.

Problema di Efficienza (Indici) Le strutture di indicizzazione spaziale (come R-Trees o KD-Trees) diventano esponenzialmente meno efficienti.

- Richiedono spazio di memorizzazione crescente.
- Il tempo di attraversamento dell'albero aumenta.
- Sopra una certa soglia (spesso $N > 10-15$), la ricerca indicizzata diventa più lenta di una scansione sequenziale dell'intero database (Sequential Scan).

Problema di Efficacia (Distanze) Più dimensioni aggiungiamo, più dati servono per evitare l'*overfitting*. Ma il problema geometrico è più grave: in spazi ad alta dimensionalità, la distribuzione delle distanze si "appiattisce". La distanza tra il punto di query e il suo vicino più prossimo (d_{min}) tende ad avvicinarsi alla distanza con il vicino più lontano (d_{max}).

$$\lim_{N \rightarrow \infty} \frac{d_{max} - d_{min}}{d_{min}} \rightarrow 0$$

Se tutti i punti sono quasi equidistanti, il concetto di "similarità" perde significato.

3.2 Effetto geometrico: Ipersfere e Gusci

Al crescere delle dimensioni, il volume dello spazio si concentra nei "gusci" esterni o negli angoli dell'iper-cubo, lasciando il centro vuoto (*Empty Space Phenomenon*). Supponiamo che lo spazio sia diviso in gusci concentrici attorno alla query. Se il numero di punti nella sfera più interna è I , il volume (e quindi il numero di punti attesi in distribuzione uniforme) nei gusci esterni cresce esponenzialmente:

- 1° strato (interno): I
- 2° strato: $\approx O(2^{n-1}I)$
- 3° strato: $\approx O(3^{n-1}I)$

Questo implica che la stragrande maggioranza dei dati si trova in uno strato periferico lontanissimo dalla query, rendendo la discriminazione basata sulla distanza estremamente difficile.

3.2.1 Power Law

Spesso la distribuzione delle distanze o la frequenza delle feature segue una legge di potenza (Power Law). Pochi termini sono molto frequenti (o poche dimensioni portano molta informazione), mentre una "coda lunga" di dimensioni aggiunge rumore o informazione marginale, contribuendo però negativamente al calcolo delle distanze (fenomeno di concentrazione della misura).

3.3 Criteri per la Selezione delle Feature

Per scegliere il sottoinsieme ottimo di feature, ci basiamo su:

1. **Semantica dell'applicazione:** Le feature devono avere senso nel dominio specifico (es. texture per tessuti, forma per loghi, colore per quadri).
2. **Percezione utente:** Se l'occhio umano non distingue due sfumature (JND - Just Noticeable Difference), è inutile usare due bin separati. La granularità deve rispecchiare la percezione.
3. **Potere descrittivo:** La feature deve descrivere l'oggetto in modo fedele.
4. **Potere discriminante:** È il criterio più importante per il retrieval. La feature deve saper distinguere oggetti diversi. Se una feature ha lo stesso valore per tutto il database, è inutile.
5. **Query Workload:** Scartiamo feature che non verranno mai interrogate dagli utenti (es. feature audio in un DB dove si cercano solo immagini per colore).

3.3.1 Caratteristiche di una "Buona Feature"

Una feature è utile se è **significativa** e **discriminante**. Esiste un legame diretto con la teoria dell'informazione: un evento (o feature) è significativo se porta informazione.

- **Evento comune:** "Oggi c'è il sole" → Bassa sorpresa → Bassa informazione.
- **Evento raro:** "Oggi c'è un'eclissi" → Alta sorpresa → Alta informazione.

Vogliamo feature che catturino eventi "rari" o distintivi degli oggetti, non caratteristiche piatte comuni a tutti.

3.4 Entropia come misura di qualità

Possiamo formalizzare il potere discriminante di una feature usando l'Entropia di Shannon $H(X)$. Data una feature X che può assumere valori x con probabilità $p(x)$:

$$H(X) = \sum_{x \in X} p(x) \log_2 \frac{1}{p(x)} = - \sum_{x \in X} p(x) \log_2 p(x)$$

- **Alta Entropia (vicino a $\log_2 N$):** La distribuzione dei valori è uniforme (istogramma piatto). La feature è molto varia e imprevedibile. **Ottima per discriminare.**
- **Bassa Entropia (vicino a 0):** La distribuzione è concentrata su un solo valore (istogramma a picco). Tutti gli oggetti hanno quasi lo stesso valore per questa feature. **Pessima per discriminare.**

3.5 Feature Latenti e Trasformazione dello Spazio

In un caso in cui ho due feature, entrambe parallele agli assi, ma devo sceglierne solo una, quale scelgo? La scelta migliore è quella che ha più potere discriminante, cioè quella con entropia più alta (in poche parole quella che se scelgo un intervallo di valori, restituisce un numero di oggetti più piccolo possibile).

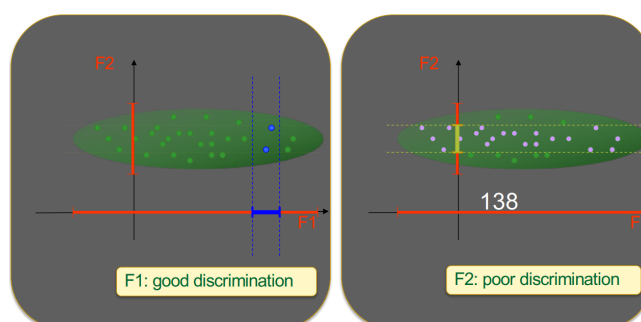


Figura 20: Esempio di scelta tra due feature parallele agli assi

Spesso le feature misurate ("native") sono ridondanti o correlate (es. altezza e peso). Questo spreca dimensioni e peggiora la dimensionality curse. L'obiettivo è trovare delle **feature latenti** (nascoste) che sintetizzino l'informazione.

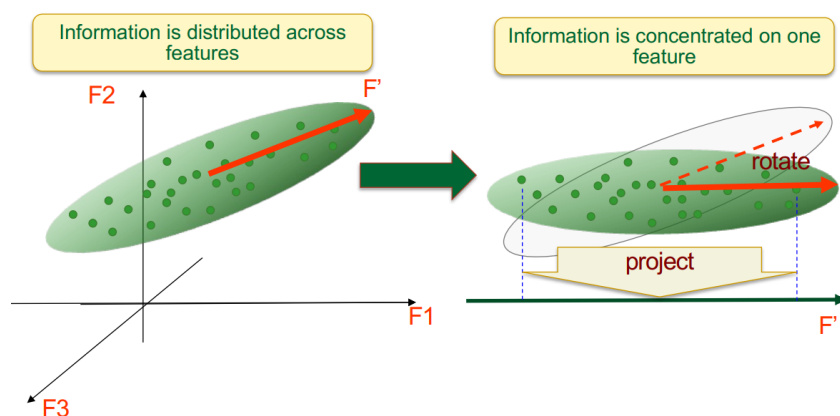


Figura 21: Esempio di feature latente

Il processo (es. PCA/SVD)

1. **Analisi della Correlazione:** Se nessuna feature è allineata con gli assi, i dati formano una nuvola obliqua.
2. **Rotazione:** Cerchiamo la direzione di *massima variabilità* dei dati. Questa direzione diventa il nuovo primo asse.
3. **Combinazione Lineare:** La nuova feature (latente) non è più una singola proprietà fisica, ma una combinazione matematica delle feature originali (es. $0.7 \cdot \text{Altezza} + 0.3 \cdot \text{Peso}$).
4. **Proiezione:** Tutti i dati (e le future query) vengono proiettati su questo nuovo sistema di riferimento.

3.5.1 Cosa sono le Feature Latenti

Le feature latenti sono proprietà **nascoste** dei dati che non vengono misurate direttamente, ma estratte matematicamente analizzando le correlazioni tra le feature originali. Mentre le feature misurate descrivono i dati grezzi, le feature latenti rappresentano i concetti astratti o le strutture fondamentali che governano quei dati, ripuliti dal rumore e dalla ridondanza.

Esempio: Dai Termini ai Concetti (Text Mining) Immaginiamo un database di documenti dove le feature misurate sono le singole parole. Se l'analisi rileva che i termini "motore", "ruote", "freni" e "carrozzeria" compaiono quasi sempre insieme (alta covarianza), significa che sono fortemente correlati.

Il sistema matematico aggrega queste variabili in un'unica feature latente. Questa nuova feature non corrisponde a una parola scritta, ma rappresenta il Topic "Automobili".

- **Prima:** Avevamo 4 dimensioni (4 parole diverse).
- **Dopo:** Abbiamo 1 dimensione (il concetto di "Automobile").

Il sistema ha ridotto la dimensionalità catturando la semantica nascosta dietro le parole.

Vantaggi

- **Compressione:** Spesso le prime k feature latenti catturano il 95% dell'informazione totale. Possiamo scartare le restanti $N - k$ dimensioni (riduzione dimensionale).
- **Decorrelazione:** Le feature latenti sono ortogonali tra loro.
- **Robustezza:** L'utente non interroga direttamente le feature latenti (non hanno un nome semantico intuitivo), ma il sistema le usa per calcolare distanze più accurate e veloci.

3.6 Rappresentazione dei testi e Feature Extraction

Nel campo dell'Information Retrieval e del Natural Language Processing, ogni documento testuale viene tipicamente modellato attraverso l'approccio **Bag of Words (BoW)**. In questo modello, un documento è rappresentato come un **multi-insieme (multi-set) di parole chiave**.

Parliamo di *multi-insieme* e non di semplice *insieme* perché viene mantenuta l'informazione sulla **frequenza** di ciascuna parola all'interno del documento. È proprio la frequenza (il numero di occorrenze) che ci permette di caratterizzare e distinguere un documento rispetto a un altro.

In questa rappresentazione strutturata **l'ordine delle parole viene ignorato**. Di conseguenza, frasi come *"il gatto insegue il topo"* e *"il topo insegue il gatto"* possiedono esattamente la stessa rappresentazione vettoriale, perdendo l'informazione legata alla sintassi.

Affinché la rappresentazione sia efficace, le parole chiave estratte devono essere **portatrici di contenuto**. È quindi fondamentale distinguere il vocabolario in due categorie:

- **Content words (Terms):** Sono le parole che veicolano il significato del testo e caratterizzano un documento (es. sostantivi, verbi, aggettivi).

Questi termini non vengono quasi mai mantenuti nella loro forma flessa originale. Per ridurre la dimensionalità del vocabolario e aggregare parole con lo stesso significato, si applicano due processi principali:

- **Stemming:** Taglia la parte finale della parola per estrarne la radice (es. "correre", "correva", "corso" → "corr").
- **Lemmatizzazione:** Riconduce la parola alla sua forma base, o *lemma*, tipicamente quella che troveremmo sul dizionario (es. maschile singolare per gli aggettivi, infinito per i verbi).

Questo processo aumenta la frequenza aggregata dei termini, migliorandone il potere discriminante. Inoltre, i sistemi più avanzati riconoscono le **parole composte** (o *n-grammi*), come "casa editrice", trattandole come un concetto unico (feature singola) anziché come la somma dei singoli termini "casa" ed "editrice". L'insieme di tutte queste feature raffinate costituisce il **vocabolario** utilizzato per costruire i vettori.

- **Non-content words (Stop words):** Sono parole di uso comune che fungono da collante sintattico ma non portano informazione semantica rilevante (es. articoli, preposizioni, congiunzioni). Essendo presenti in modo omogeneo in quasi tutti i documenti della collezione, non aiutano a discriminare un testo dall'altro. Dal punto di vista della Teoria dell'Informazione, avendo una probabilità di comparsa altissima, queste parole apportano un **contenuto informativo (self-information) quasi nullo**. Per questo motivo, vengono filtrate e rimosse in fase di pre-processing.

3.6.1 Rappresentazione vettoriale (Vector Space Model)

Dato un vocabolario di termini estratti dalla collezione, ogni documento d_i viene rappresentato come un vettore numerico:

$$d_i = \langle w_{i1}, w_{i2}, w_{i3}, \dots, w_{in} \rangle$$

La dimensione del vettore corrisponde all'esatto numero di termini n presenti nell'intero vocabolario.

Il peso w_{ij} quantifica l'importanza (o la frequenza) del termine j -esimo all'interno del documento i -esimo:

- $w_{ij} = 0$: se il termine j **non compare** nel documento d_i . Spesso questi vettori sono *sparsi*, contenendo moltissimi zeri.
- $w_{ij} > 0$: se il termine j **compare** nel documento d_i , con un valore che cresce in base alla sua rilevanza.

3.6.2 Distribuzione Zipfiana

La distribuzione delle frequenze dei termini in un linguaggio naturale (e quindi nei documenti) non è uniforme, ma segue la **Legge di Zipf**. Essa stabilisce che la frequenza della k -esima parola più comune in una collezione è inversamente proporzionale al suo rango k , ed è pari a $(\frac{1}{k})^\Theta$ volte la frequenza della parola più comune (con il parametro $\Theta \approx 1$).

Questo significa che poche parole occorrono spessissimo (le *stop words*), mentre la stragrande maggioranza dei termini compare molto raramente.

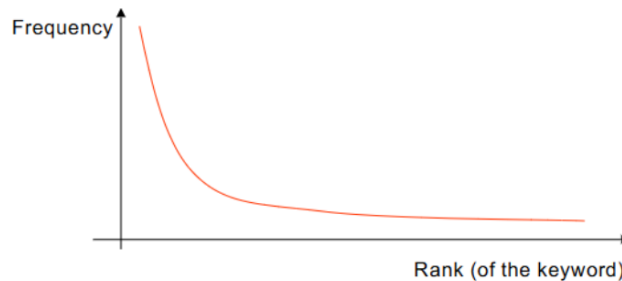


Figura 22: Rappresentazione grafica della Legge di Zipf: si nota il rapido decadimento della frequenza all'aumentare del rango del termine.

3.6.3 Come definire i pesi vettoriali

I pesi vettoriali devono quantificare quanto una determinata feature sia "brava" a descrivere il contenuto di uno specifico documento all'interno di una collezione.

1. Term Frequency (TF) La *Term Frequency* è una misura locale che indica la densità di un termine all'interno di un *singolo* documento. È definita come:

$$tf = \frac{n}{K}$$

dove n è il numero di occorrenze del termine nel documento e K è il numero totale di termini in quel documento.

Esempio: Se in un documento breve la parola "Partita" compare 3 volte, e in un documento molto lungo compare 30 volte, dividere per K (normalizzazione sulla lunghezza del documento) ci permette di capire in quale dei due documenti il concetto di "Partita" è in proporzione più centrale e rilevante.

Tuttavia, la TF da sola non basta per valutare il **potere discriminante** di una feature. Se una parola è frequente in un documento, ma è ugualmente frequente in *tutti* gli altri documenti della collezione, allora non ci aiuta a distinguerli. Al contrario, un termine raro nella collezione generale, ma molto frequente nel nostro documento, risulta essere un'ottima feature distintiva.

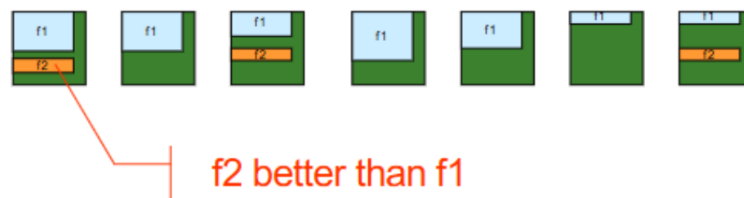


Figura 23: Esempio intuitivo: una feature presente in pochi documenti ha un potere discriminante molto più elevato rispetto a una feature ubiquitaria.

2. Inverse Document Frequency (IDF) L'*Inverse Document Frequency* è una misura globale che valuta quanto un termine sia raro o comune in **tutta la collezione** di documenti:

$$idf = \log \left(\frac{N}{m} \right)$$

dove N è il numero totale di documenti della collezione e m è il numero di documenti in cui compare almeno una volta il termine in esame. Più il termine è raro (ovvero m è piccolo), maggiore sarà il valore di $\frac{N}{m}$, e di conseguenza l'IDF sarà più alto.

Nota architetturale: Poiché l'IDF dipende da N (che cambia man mano che si inseriscono nuovi documenti nel sistema), i motori di ricerca e i database documentali solitamente non memorizzano l'IDF staticamente, ma aggiornano o calcolano questo valore dinamicamente.

3. TF-IDF Il peso finale più utilizzato è il prodotto tra TF e IDF, chiamato **TF-IDF**. Questa metrica bilancia l'importanza locale del termine e la sua specificità globale:

$$tf_idf = tf \cdot idf = \frac{n}{K} \cdot \log\left(\frac{N}{m}\right)$$

Interpretazione: Due termini che compaiono con la stessa frequenza (TF) in un documento avranno pesi finali (TF-IDF) diversi: il termine presente in meno documenti della collezione (IDF più alto) riceverà un peso maggiore perché è più caratteristico.

Gestione degli edge cases (Smoothing): Un potenziale problema della formula standard dell'IDF si presenta quando $m = 0$ (un termine cercato non è presente in nessun documento della collezione), portando il denominatore a zero e generando un asintoto a infinito. Per risolvere questo problema, nei sistemi reali si utilizza spesso una versione "lisciata" (smoothed), aggiungendo 1 al denominatore: $idf = \log\left(\frac{N}{m+1}\right)$.

4. Normalizzazione del TF-IDF La componente TF ($\frac{n}{K}$) assume per sua natura valori compresi tra 0 e 1. L'IDF, invece, è un logaritmo che può crescere notevolmente per termini estremamente rari in collezioni immense. Per evitare che il fattore globale (IDF) "schiacci" completamente il fattore locale (TF), si adotta spesso una versione **normalizzata** della metrica:

$$norm_tf_idf = \frac{n}{K} \cdot \frac{\log\left(\frac{N}{m}\right)}{\max idf}$$

Dividendo per il valore massimo di IDF calcolato nella collezione, si forza anche il termine IDF in un range tra 0 e 1. In questo modo si dà pari potenziale alle due componenti. Nel caso in cui si desideri asimmetria (es. privilegiare la rarità globale), si possono inserire dei coefficienti di peso correttivi (α, β).

3.7 Estrazione di Feature dalle Immagini

Per poter analizzare e confrontare le immagini tramite algoritmi, è necessario passare dalla matrice grezza di pixel a una **rappresentazione vettoriale** (feature vector).

Una rappresentazione intermedia molto comune è l'utilizzo degli **istogrammi**, che riassumono la distribuzione globale di una certa caratteristica nell'immagine. Un istogramma può essere interpretato come un conteggio delle frequenze di valori esatti, oppure come l'accumulo di valori all'interno di intervalli predefiniti (*bin*).

Le feature estraibili da un'immagine possono essere classificate in base al loro livello di astrazione:

- **Low-level features:** Colori, Edges (bordi/contorni), Texture (trama).
- **Mid-level features:** Segments (regioni o segmenti di immagine con caratteristiche omogenee).
- **High-level features:** Objects (oggetti riconosciuti), Metadata (informazioni EXIF), Captions e Tags.

3.7.1 Rappresentazione del Colore

Modello RGB Nel modello **RGB**, ciascun colore è ottenuto tramite la sintesi additiva di tre componenti fondamentali: Rosso (R), Verde (G) e Blu (B). Queste componenti sono considerate *linearmente indipendenti*, ovvero nessuna delle tre può essere ottenuta come combinazione matematica delle altre due. Fungono da "vettori base" per generare tutti gli altri colori dello spettro visibile a monitor.

Lavorando tipicamente su 8 bit per canale, i valori spaziano da 0 a 255: la terna (0, 0, 0) rappresenta il nero (assenza totale di luce), mentre (255, 255, 255) rappresenta il bianco (massima intensità luminosa di tutti i canali).

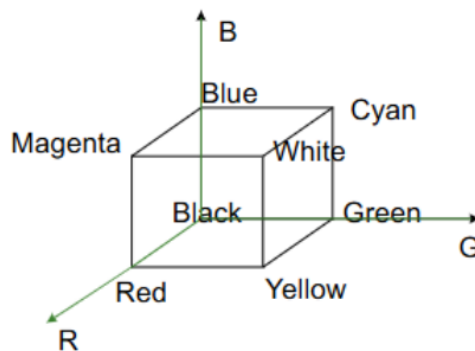


Figura 24: Scomposizione di un'immagine nei tre canali del modello RGB.

Modello HSV Se l'obiettivo è elaborare l'immagine basandosi su come gli esseri umani "pensano" al colore, il modello RGB risulta poco intuitivo. Il modello **HSV** separa la componente cromatica da quella di illuminazione, risultando molto più vicino alla percezione umana:

- **Hue (H - Tonalità)**: Rappresenta il tipo di colore (es. rosso, verde, blu). Geometricamente è spesso rappresentato come un angolo (da 0° a 360°) su una ruota dei colori.
- **Saturation (S - Saturazione)**: Indica la purezza o "pienezza" del colore (da 0 a 1). A 0 si ha una sfumatura di grigio, a 1 si ha il colore puro senza contaminazioni di bianco.
- **Value (V - Valore/Luminosità)**: Indica la brillantezza del colore (da 0 a 1). A 0 il colore è completamente nero, a 1 ha la massima luminosità possibile.

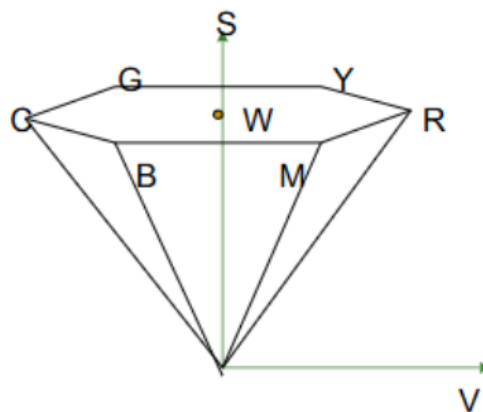


Figura 25: Rappresentazione cilindrica e scomposizione dello spazio colore HSV.

Modello CIELAB (o Lab*) Il modello CIELAB è uno spazio colore *perceptivamente uniforme*. Questo significa che una variazione numerica fissa all'interno di questo spazio corrisponde a una variazione visiva di pari entità percepita dall'occhio umano, cosa che non avviene in RGB o HSV. È lo standard per misurare le differenze oggettive tra colori.

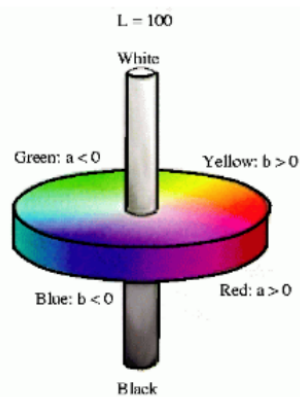


Figura 26: Esempio di rappresentazione nello spazio colore uniforme CIELAB.

3.7.2 Rappresentazioni Lossy e Compressione

Un'immagine grezza ad alta risoluzione contiene spesso micro-dettagli o variazioni cromatiche che l'occhio umano non è fisicamente in grado di percepire. Utilizzare un modello troppo dettagliato, mantenendo tutte queste informazioni, è controproducente: aumenta drasticamente lo spazio di archiviazione, i tempi di calcolo, e rischia di far distinguere all'algoritmo feature che in realtà sono semplice "rumore" visivo, peggiorando il potere discriminante del modello.

Per questo si ricorre a tecniche di **riduzione della dimensionalità** e compressione **lossy** (con perdita di dati, non reversibile, come il formato JPEG).

Il principio biologico fondamentale su cui si basano queste compressioni è che **l'occhio umano è molto più sensibile alle variazioni di luminosità (contrasto) che alle variazioni di colore**.

Modello YUV Per sfruttare questo limite biologico, si passa dallo spazio RGB a spazi colore che separano la luminosità dal colore, come lo **YUV** (una trasformazione lineare di RGB):

- **Y (Luma)**: Rappresenta la luminanza, ovvero l'immagine in scala di grigi. Contiene il dettaglio e il contrasto dell'immagine.
- **U e V (Chroma)**: Rappresentano la cromaticanza (l'informazione sul colore). In particolare, *U* mappa l'asse blu-giallo e *V* l'asse rosso-ciano.

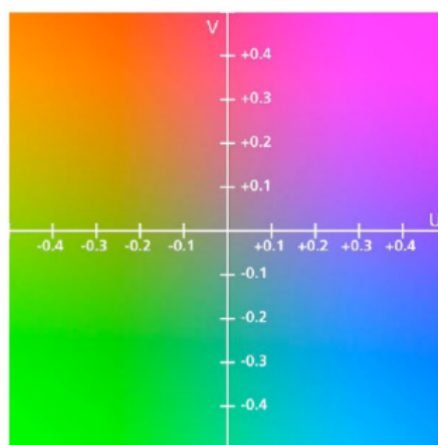


Figura 27: Schema del piano colore YUV.

Poiché la componente Y è la più importante per l'occhio umano, in fase di compressione possiamo preservarla al massimo e "sacrificare" le componenti U e V (tecnica nota come *Chroma Subsampling*).

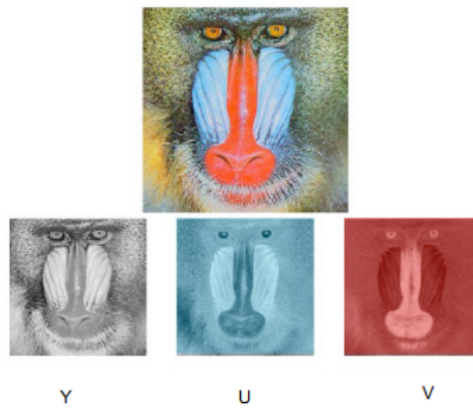


Figura 28: Scomposizione di un'immagine nei canali Y, U e V. Si nota come il canale Y contenga quasi tutta l'informazione strutturale.

Esempio pratico: Se in RGB usiamo 8 bit per canale (24 bit totali per pixel), non possiamo scartare bit senza alterare drasticamente l'immagine, poiché i tre canali sono ugualmente importanti. In YUV, potremmo decidere di usare 8 bit per la componente Y (mantenendo intatti i dettagli di luce) e solo 4 bit per U e 4 bit per V, portando il peso del pixel da 24 a 16 bit totali senza che l'occhio umano percepisca un degrado significativo della qualità.

Modello YIQ Oltre a essere più sensibile alla luminosità che al colore, il sistema visivo umano ha un'ulteriore particolarità: discrimina i dettagli molto meglio nelle sfumature tra l'arancione e il blu rispetto a quelle tra il verde e il magenta. Il modello **YIQ** (storicamente usato per le trasmissioni televisive NTSC) sfrutta proprio questo:

- **Y:** Luminanza (scala di grigi).
- **I (In-phase):** Rappresenta l'asse cromatico arancione-blu (a cui siamo più sensibili).
- **Q (Quadrature):** Rappresenta l'asse cromatico verde-magenta (a cui siamo meno sensibili).

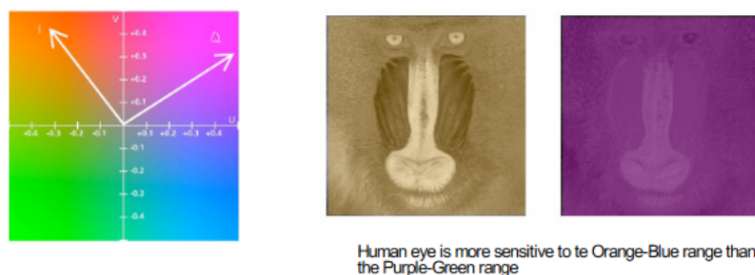


Figura 29: Rappresentazione dei canali nello spazio colore YIQ.

Esempio di compressione estrema: In fase di codifica, conoscendo questi limiti biologici, potremmo assegnare 8 bit al canale Y (massima fedeltà per i contorni), 6 bit al canale I (buona fedeltà per i colori arancione/blu) e soli 2 bit al canale Q (bassissima fedeltà per verde/magenta). In questo modo, l'immagine viene fortemente compressa pur mantenendo un'apparenza visiva eccellente per un osservatore umano.

Momenti dei colori I momenti dei colori sono feature globali compatte che descrivono statisticamente la distribuzione dei valori di colore all'interno di un'immagine. Dato un canale colore (ad esempio, il rosso R), i primi tre momenti sono:

- **Primo momento (Media - Mean):**

$$Avg_R = \frac{1}{N} \sum_{x,y} R(x,y)$$

dove N è il numero totale di pixel. Rappresenta l'intensità media di quel colore nell'immagine. Un Avg_R alto indica un'immagine prevalentemente rossa.

- **Secondo momento (Deviazione standard - Standard Deviation):**

$$STD_R = \sqrt{\frac{1}{N} \sum_{x,y} (R(x,y) - Avg_R)^2}$$

Misura la varianza, ovvero quanto i pixel si discostano in media dal valore centrale. Un valore alto indica un elevato contrasto o una grande varietà di sfumature di quel colore (es. un tramonto); un valore basso indica un colore molto uniforme (es. un cielo sereno o uno sfondo a tinta unita).

- **Terzo momento (Asimmetria - Skewness):**

$$Skew_R = \sqrt[3]{\frac{1}{N} \sum_{x,y} (R(x,y) - Avg_R)^3}$$

Indica l'asimmetria della distribuzione dei colori rispetto alla media.

- Se è **positivo**, la distribuzione ha una "coda" verso i valori alti: la maggior parte dei pixel ha un valore di rosso inferiore alla media, ma ci sono pochi pixel con rosso molto intenso.
- Se è **negativo**, la coda è verso i valori bassi: l'immagine ha molti pixel di rosso intenso, con pochi pixel scuri che abbassano la media.

Si potrebbe proseguire con momenti di ordine superiore (es. la *Curtosi*), ma in pratica i primi tre momenti sono sufficienti a fornire un'ottima sintesi. Poiché l'immagine ha tipicamente 3 canali (es. RGB o HSV), un vettore di feature basato sui momenti avrà dimensione $3 \times 3 = 9$ feature in totale.

Essendo misure globali, i momenti perdono completamente l'informazione sulla **disposizione spaziale** dei colori. Per questo, spesso si integrano con istogrammi o partizioni locali.

3.7.3 Istogramma dei colori

Un istogramma rappresenta la distribuzione globale dei colori quantizzati. Per ogni "bin" (che rappresenta un colore o un range di colori del vocabolario scelto), l'istogramma conta il numero assoluto o la percentuale di pixel che assumono quel colore nell'immagine.

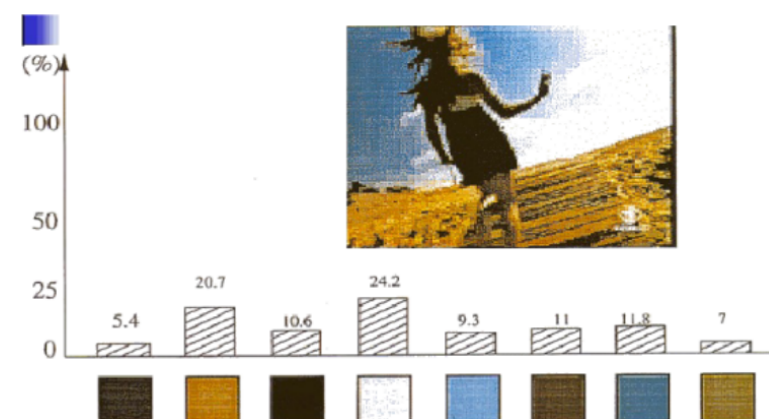


Figura 30: Esempio di istogramma dei colori: l'asse X rappresenta i colori del vocabolario, l'asse Y la frequenza dei pixel.

Problemi degli istogrammi standard Il limite principale degli istogrammi standard è la mancanza di correlazione semantica tra i bin. Il conteggio considera ogni colore come indipendente: un pixel rosso fuoco e un pixel rosso corallo, se assegnati a due bin diversi, vengono considerati distanti quanto un pixel rosso e uno blu. Questo problema viene risolto utilizzando distanze avanzate (come la forma quadratica) che considerano il *cross-talk* tra i colori.

3.7.4 Istogrammi di colore locali (Spatial Grids)

Per recuperare l'informazione spaziale persa dagli approcci globali, l'immagine viene frammentata in una griglia fissa (es. 3×3 o 4×4 celle). Per ogni cella viene calcolato un istogramma (o i momenti dei colori) indipendente. Il vettore di feature finale è la concatenazione delle feature locali. In questo modo, il sistema è in grado di distinguere, ad esempio, un'immagine con il "cielo azzurro in alto e prato verde in basso" da una con "prato verde in alto e mare azzurro in basso", che globalmente avrebbero lo stesso istogramma.

3.7.5 Come scegliere il vocabolario dei colori

Per costruire un istogramma è necessario discretizzare lo spazio continuo dei colori in un numero finito di *bin*. Esistono due approcci principali:

1. Vocabolario definito a priori (Data-independent) Lo spazio colore viene partizionato staticamente, indipendentemente dal dataset. Ad esempio, si dividono uniformemente gli assi R, G e B in 4 intervalli ciascuno, ottenendo $4^3 = 64$ bin.

- **Pro:** Sistema flessibile, pronto ad accogliere qualsiasi nuova immagine senza dover ricalcolare il modello.
- **Contro:** Molti bin potrebbero rimanere vuoti (es. se nel database non ci sono immagini viola, il bin del viola sarà un inutile spreco di spazio).

2. Vocabolario definito dai dati (Data-driven) Se il database è relativamente statico e si conosce il dominio delle immagini, si analizza la distribuzione reale dei pixel in tutto il dataset per creare un vocabolario su misura. In questo modo si ottimizza lo spazio: ci saranno più bin dedicati ai colori dominanti del dataset e pochissimi per i colori rari.

Algoritmo Median-Cut È l'algoritmo classico per la quantizzazione data-driven dei colori. Si basa su una divisione iterativa dello spazio RGB:

1. Si inseriscono tutti i pixel del database in un unico grande contenitore (cluster).
2. Si individua il contenitore con la massima varianza lungo uno dei tre assi colore (R, G o B). Ad esempio, si trova che i colori variano maggiormente lungo il canale del Verde.
3. Si ordinano i pixel di quel contenitore rispetto al canale scelto e si calcola il valore **mediano**.
4. Si "taglia" il contenitore in due esatte metà basandosi su questo valore mediano, creando due nuovi cluster.
5. Si ripete il processo sui nuovi cluster fino a raggiungere il numero di bin desiderato (K).

Il risultato è un vocabolario ottimizzato dove ogni bin ha quasi lo stesso numero di pixel complessivi, garantendo un'alta entropia e un ottimo potere discriminante.

3.7.6 Confronti tra istogrammi dei colori

Siano H e K due istogrammi normalizzati (frequenze relative) con n bin.

- **Distanza Euclidea** (L_2 norm):

$$D_{Euc}(H, K) = \sqrt{\sum_{i=1}^n (H_i - K_i)^2}$$

Semplice e veloce, ma non tiene conto della somiglianza tra colori di bin diversi (problema dell'indipendenza dei bin).

- **Intersection Similarity (Istogram Intersection):**

$$S_{Int}(H, K) = \frac{\sum_{i=1}^n \min(H_i, K_i)}{\sum_{i=1}^n K_i}$$

Misura l'area di sovrapposizione tra i due istogrammi. Se gli istogrammi sono identici, il valore è 1. È molto robusta ai rumori di fondo e occlusioni parziali.

Distanza in Forma Quadratica (Quadratic Form Distance) Per risolvere il problema dell'indipendenza dei bin, si utilizza una matrice di pesi A .

$$D_{quad}^2(H, K) = (H - K)^\top A (H - K) = \sum_{i=1}^n \sum_{j=1}^n A_{ij} (H_i - K_i)(H_j - K_j)$$

Il fattore A_{ij} è detto *cross-talk* e quantifica la somiglianza percettiva tra il colore del bin i e quello del bin j .

- Se A è la matrice identità (tutti zeri tranne la diagonale), la formula si riduce alla Distanza Euclidea standard.
- Se A_{ij} è alto, l'algoritmo "perdona" l'immagine se un pixel è caduto nel bin del rosso chiaro anziché in quello del rosso scuro, considerandoli affini.

Problema: La complessità computazionale diventa quadratica $O(n^2)$. Confrontare istogrammi molto densi su database enormi diventa troppo lento.

Filtro spaziale e Lower-Bounding (Filter-and-Refine) Per accelerare la ricerca senza perdere precisione, si applica la teoria del *Lower Bounding* (alla base di algoritmi come GEMINI).

Consideriamo le medie dei canali RGB delle due immagini come un vettore a 3 dimensioni: $\mu_x = [R_{avgx}, G_{avgx}, B_{avgx}]$ e $\mu_y = [R_{avgy}, G_{avgy}, B_{avgy}]$. È possibile dimostrare matematicamente che esiste una costante c tale per cui la distanza euclidea tra le medie è sempre inferiore o uguale alla complessa distanza quadratica tra gli istogrammi:

$$d_{Euc}^2(\mu_x, \mu_y) \leq c \cdot D_{quad}^2(H_x, H_y)$$

Questa disuguaglianza ci permette di adottare un'architettura a due fasi:

1. **Filtro (Filter):** Si scansiona il database calcolando solo la velocissima distanza tra i momenti medi. Se questa distanza "sottostimata" è già superiore a una nostra soglia di tolleranza ϵ , possiamo scartare l'immagine con assoluta certezza matematiche (Zero Falsi Negativi o *No False Dismissals*).
2. **Raffinamento (Refine):** Si recupera il piccolo sottoinsieme di immagini "sopravvissute" al filtro e solo su queste si calcola la pesante distanza in forma quadratica per stilare il ranking definitivo.

Questo stratagemma abbatte drasticamente i tempi di calcolo garantendo esattamente gli stessi risultati di una scansione quadratica completa.

3.7.7 Texture

La texture (o trama) descrive la presenza di pattern regolari o quasi-regolari che si ripetono all'interno di una regione dell'immagine. A differenza del colore, che è una proprietà puntuale del singolo pixel, la texture è una **proprietà spaziale e di vicinato**: non ha senso calcolare la texture di un singolo pixel isolato.

Esistono molteplici proprietà percettive per descrivere le texture, tra cui:

- Livello di grigio (luminosità media del pattern)
- Forma degli elementi base (texton)
- Omogeneità e Levigatezza

- Finezza e Granulosità
- Ruvidezza
- Regolarità e Periodicità
- Linearità e Direzionalità

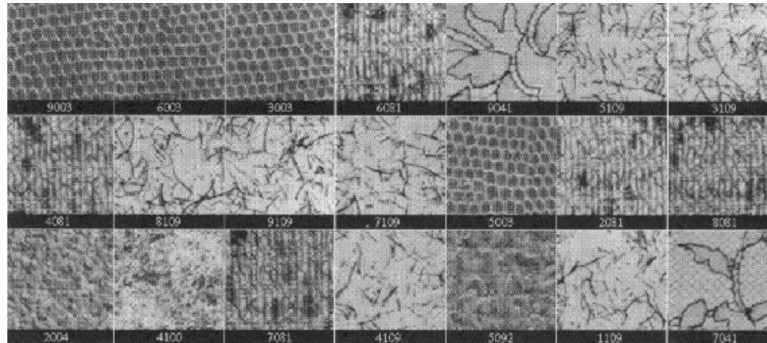


Figura 31: Esempi di texture con differenti gradi di direzionalità, ruvidezza e regolarità.

Estrazione dei Gradienti (Filtri di Sobel) Per analizzare la texture, il primo passo è tipicamente la conversione dell'immagine in **scala di grigi**, poiché l'informazione strutturale risiede nei contrasti di luminosità (luma) piuttosto che nelle variazioni cromatiche.

L'obiettivo è misurare la **direzione** e l'**intensità** (magnitudo) della variazione di luminosità per ogni pixel rispetto ai suoi vicini. Questo si ottiene calcolando il gradiente spaziale dell'immagine tramite l'operazione di **convoluzione**. La convoluzione applica una trasformazione lineare locale utilizzando una matrice di pesi detta *kernel* o *filtro*, facendola scorrere pixel per pixel.

I filtri più utilizzati per questo scopo sono i **filtri di Sobel**, che approssimano le derivate parziali dell'immagine lungo l'asse orizzontale (x) e verticale (y):

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \quad G_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Il filtro G_x calcola il gradiente orizzontale (enfaticizzando i bordi verticali), assegnando un peso maggiore ai pixel immediatamente a destra (+2) e a sinistra (-2) del pixel centrale. Analogamente, G_y calcola il gradiente verticale.

Per ogni pixel, le due convoluzioni restituiscono due valori scalari, δ_x e δ_y . Da questi, calcoliamo le due componenti fondamentali del gradiente:

- **Intensità (Magnitudo)** della variazione:

$$|\nabla| = \sqrt{\delta_x^2 + \delta_y^2}$$

- **Direzione** della variazione (fase):

$$\theta = \arctan\left(\frac{\delta_y}{\delta_x}\right)$$

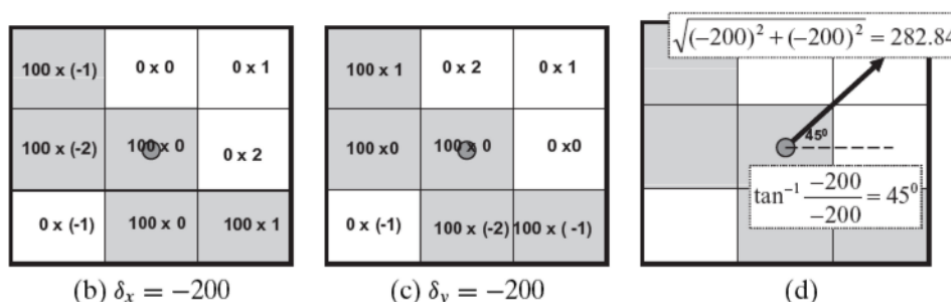


Figura 32: Esempio di estrazione dei gradienti di un'immagine in scala di grigi utilizzando i filtri di Sobel.

Istogramma delle Texture e dei Gradienti A differenza dell'istogramma dei colori, dove i "bin" sono i colori stessi, qui abbiamo valori continui (reali) per magnitudo e fase. Per costruire una rappresentazione vettoriale sintetica, dobbiamo **quantizzare** queste misure.

Esistono diversi approcci per costruire istogrammi basati su queste informazioni spaziali:

1. **Histogram of Oriented Gradients (HOG)**: Si discretizza lo spazio degli angoli θ (es. 8 bin da 0° a 180°). Ogni pixel "vota" per il bin corrispondente alla sua direzione θ , e il peso del voto è proporzionale alla sua intensità $|\nabla|$. Questo crea un istogramma che descrive in quali direzioni si concentrano i bordi o i pattern della texture.
2. **Matrice di Co-occorrenza (GLCM)**: Cattura la relazione spaziale esatta tra coppie di pixel. Si fissa una distanza d e un angolo α , e si conta quante volte un pixel con intensità di grigio i è adiacente a un pixel di intensità j lungo quella direzione. È concettualmente un istogramma multidimensionale molto potente per texture complesse, ma computazionalmente oneroso.

Nota sull'efficienza: mentre l'istogramma dei colori si calcola con un singolo passaggio lineare $O(N)$, l'estrazione delle feature di texture richiede operazioni di convoluzione e spesso l'analisi di vicinati complessi, portando a costi computazionali significativamente maggiori.

3.8 Segmentazione e Riconoscimento delle Forme

Estrazione dei Bordi (Edge Detection) I bordi rappresentano i confini fisici o semantici tra oggetti diversi o tra un oggetto e lo sfondo. Matematicamente, un bordo corrisponde a una regione in cui il gradiente dell'immagine (calcolato ad esempio con Sobel o Canny) presenta un picco o un forte contrasto locale.

Una volta individuati i pixel appartenenti ai bordi, l'obiettivo è aggregarli in strutture coerenti (forme, linee, segmenti) da analizzare e confrontare. Possiamo procedere secondo due filosofie:

- **Metodo Analitico (Parametrico)**: Ricerca di forme geometriche note (rette, cerchi, ellissi) sfruttando le loro equazioni matematiche.
- **Metodo Descrittivo (Non Parametrico)**: Rappresentazione della forma esatta del contorno così come si presenta, indipendentemente dalla sua regolarità.

Trasformata di Hough (Hough Transform) La Trasformata di Hough è l'algoritmo classico per il **metodo analitico**. Dato un insieme di pixel sparsi che descrivono un potenziale bordo, l'obiettivo è trovare la forma geometrica (es. la retta) che passa per il maggior numero di essi, mitigando rumore e interruzioni del tratto.

Data l'equazione della retta $y = mx + q$, un singolo pixel di coordinate note (x_i, y_i) vincola i parametri m e q secondo la relazione:

$$q = y_i - x_i m$$

Questo significa che un *punto* nello spazio dell'immagine (x, y) corrisponde a una *retta* nello spazio dei parametri (m, q) . Per trovare la retta migliore, si utilizza uno **spazio di accumulazione** (una matrice di voto):

1. Per ogni punto del bordo (x_i, y_i) , si calcolano tutti i possibili valori di q al variare di m .
2. Si incrementa di 1 la cella della matrice corrispondente alla coppia (m, q) .
3. Il picco massimo nella matrice (la cella con più voti) indica i parametri (m, q) della retta che interpola il maggior numero di punti nell'immagine.

Problema di cardinalità: La pendenza m può tendere a infinito (per rette verticali), rendendo impossibile creare una matrice finita. Per questo, si utilizza la **forma polare** della retta:

$$\rho = x \cos \theta + y \sin \theta$$

dove ρ è la distanza della retta dall'origine e θ è l'angolo del vettore normale. Poiché $\theta \in [0, 180^\circ]$ e $\rho \in [-D, D]$ (con D diagonale dell'immagine), lo spazio dei parametri è chiuso e limitato, rendendo la matrice di voto computabile.

Segmentazione basata su Cluster (K-Means) La segmentazione divide l'immagine in regioni internamente coerenti. L'approccio per clustering considera ogni pixel come un punto in uno spazio multidimensionale (es. uno spazio a 5 dimensioni composto da: coordinata X , coordinata Y , e i 3 canali colore R, G, B). Applicando algoritmi come il K-Means, i pixel vengono raggruppati in base alla loro vicinanza spaziale e cromatica simultaneamente, generando segmenti compatti e dal colore omogeneo.

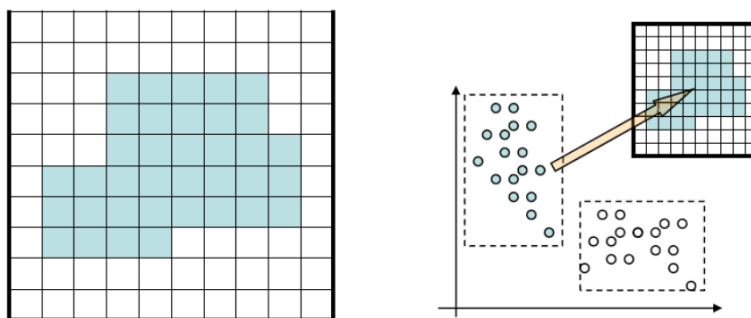


Figura 33: Esempio di segmentazione basata su clustering, dove pixel spazialmente vicini e cromaticamente simili vengono raggruppati.

Region-Growing È un approccio *bottom-up*. Si parte dalla scelta di uno o più pixel "seme" (seed points). L'algoritmo esamina i pixel adiacenti al seme: se soddisfano un determinato **criterio di omogeneità** (es. la differenza di colore o di grigio è inferiore a una soglia ϵ), vengono annessi alla regione. Il processo continua iterativamente e la regione si "espande" finché non si incontrano pixel che violano il criterio (es. bordi netti).

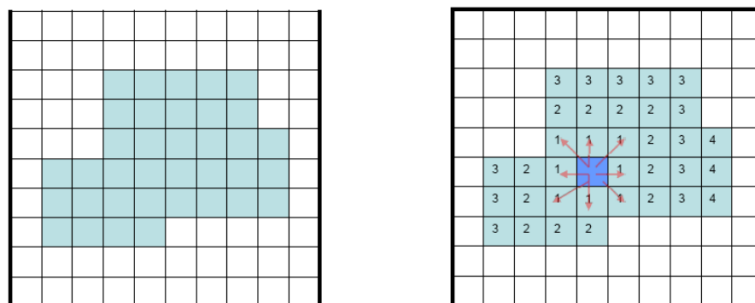


Figura 34: Esempio di segmentazione basata sul region-growing a partire da pixel seme.

Watershed (Spartiacque) Il metodo Watershed tratta l'immagine dei **gradienti** come se fosse una mappa topografica tridimensionale. In questa analogia geometrica:

- **Valli (bacini):** Regioni in cui il gradiente è vicino a zero, corrispondenti alle zone piatte e omogenee dell'immagine originale.
- **Rilievi (montagne):** Pixel con gradiente elevato, corrispondenti ai bordi tra gli oggetti.

Si immagina di "allagare" questa topografia facendo sgorgare acqua dai punti più bassi (i minimi locali del gradiente). Man mano che l'acqua sale, i bacini adiacenti rischiano di fondersi. Per impedirlo, si costruiscono delle "dighe" lungo i crinali montuosi. Al termine dell'allagamento, le dighe create rappresentano i confini esatti dei segmenti (le linee di spartiacque).

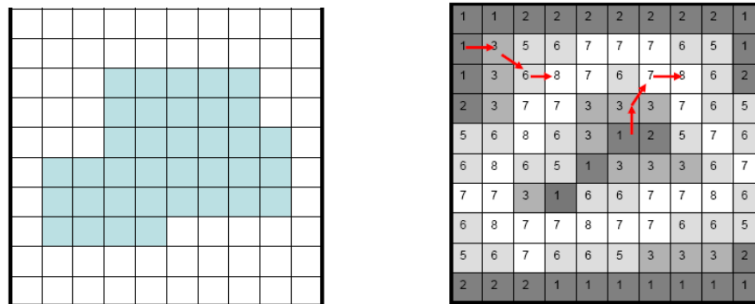


Figura 35: Esempio di segmentazione basata su watershed, con individuazione delle linee di spartiacque sui bordi.

3.9 Rappresentazione e Descrittori di Forma

Una volta isolata una forma (un segmento o un oggetto), dobbiamo tradurla in una feature vettoriale per poterla confrontare con altre forme nel database.

Rappresentazione Bitmap-based (Rigida) La forma è rappresentata mediante una matrice binaria delle stesse dimensioni dell'immagine originale, dove i pixel appartenenti all'oggetto valgono 1 e quelli dello sfondo valgono 0. Il confronto tra due forme avviene misurando l'area di sovrapposizione (es. tramite un'operazione logica XOR).

Limiti: È una rappresentazione estremamente fragile.

- **Rotazione:** Fortemente sensibile. Ruotare l'oggetto genera una matrice completamente diversa.
- **Traslazione:** Fortemente sensibile. Spostare l'oggetto cambia le coordinate degli 1.
- **Scalatura:** Fortemente sensibile. Ingrandire l'oggetto altera il conteggio e la posizione dei pixel attivi.

0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	1	1	0	0	0
0	0	0	1	1	0	1	1	0	0
0	0	1	1	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	0	1
1	1	0	0	0	0	1	1	1	0
0	1	1	1	1	1	1	0	0	0
0	0	1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Figura 36: Esempio di segmentazione e mascheratura basata su bitmap binaria.

Chain-Code (Codice a Catena di Freeman) È un descrittore basato esclusivamente sul **contorno** dell'oggetto. A partire da un pixel di inizio (spesso scelto in modo arbitrario), si percorre il perimetro dell'oggetto registrando ad ogni passo la **direzione** intrapresa per raggiungere il pixel di contorno successivo. Le direzioni sono quantizzate in un intorno discreto (tipicamente a 4 o 8 connessioni, con valori da 0 a 7). La forma viene così compressa in una **stringa numerica** monodimensionale (es. 0007766544...).

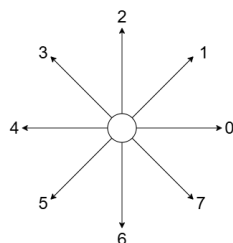


Figura 37: Codifica a 8 direzioni per la generazione del Chain-code.

Proprietà di invarianza:

- **Traslazione: Invariante.** Poiché si registrano solo gli spostamenti *relativi* da un pixel all'altro e non le coordinate spaziali assolute, traslare l'oggetto non altera in alcun modo la stringa risultante.
- **Scalatura: Sensibile.** Un oggetto più grande genererà un contorno con più pixel e quindi una stringa più lunga (es. un segmento orizzontale lungo 2 pixel è codificato come "00", uno lungo 4 pixel come "0000"). Può essere reso invariante tramite tecniche di ricampionamento spaziale del contorno a lunghezza fissa.
- **Rotazione: Sensibile** nella sua forma base, poiché ruotando l'oggetto cambiano le direzioni assolute. Tuttavia, calcolando la **derivata prima** del chain-code (ovvero registrando le variazioni di direzione tra un passo e il successivo, anziché la direzione assoluta) e allineando la stringa in modo che formi il numero intero di valore minimo, la rappresentazione diventa invariante alla rotazione per step discreti (es. multipli di 45°).

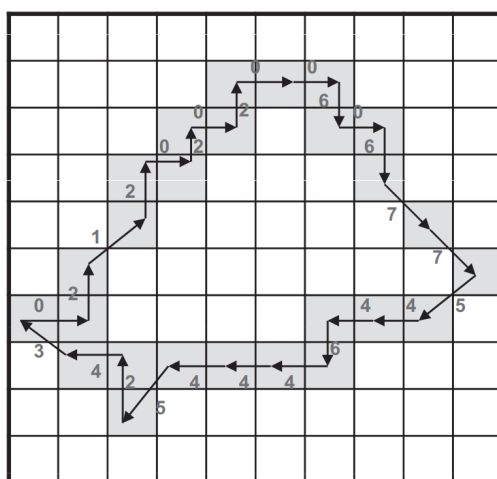


Figura 38: Esempio di estrazione del contorno e rappresentazione tramite chain-code.

Approssimazione Poligonale (Rappresentazione vettoriale) Una versione alternativa per descrivere i contorni si basa sull'idea di approssimare la forma vedendola come una concatenazione di segmenti di retta (poligonale). L'algoritmo valuta la distanza tra il contorno reale e il segmento che ne unisce gli estremi: se per tutti i pixel questa distanza (l'errore) è inferiore a una soglia prestabilita, la forma viene approssimata con una singola linea retta. Se l'errore supera la soglia, si inserisce un nuovo punto di controllo intermedio (vertice) e si divide l'approssimazione in due o più segmenti (tecnica di *split-and-merge*).

Proprietà di invarianza:

- **Traslazione: Invariante.** La posizione assoluta dei vertici non è vincolante, conta solo la loro disposizione relativa (lunghezze e angoli interni).
- **Rotazione e Scalatura:** Di base **sensibili** se guardiamo le coordinate, ma le variazioni sono facilmente **calcolabili e normalizzabili**. Se in due forme i segmenti mantengono le stesse proporzioni relative di lunghezza e gli stessi angoli interni, possiamo dedurre matematicamente che si tratta di versioni ruotate o scalate dello stesso oggetto.

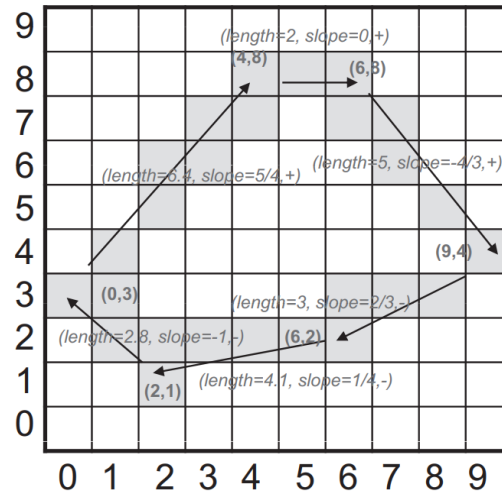


Figura 39: Esempio di approssimazione del contorno tramite vettori poligonali.

Time-Series (Firma Centroidale / Centroid Distance Signature) Questo metodo riduce drasticamente la dimensionalità trasformando il contorno bidimensionale di una forma in un segnale monodimensionale periodico (una serie temporale).

Il processo funziona così:

1. Si calcola il baricentro spaziale (centroide) dell'oggetto, che avrà coordinate (x_c, y_c) .
2. Si fa "spazzare" un raggio dal centroide verso il contorno per un angolo θ che va da 0 a 360° (2π).
3. Per ogni angolo θ , si misura la distanza euclidea $d(\theta)$ tra il centroide e il punto di intersezione sul contorno.
4. Si riporta questa misurazione su un grafico cartesiano, dove l'asse X è l'angolo θ e l'asse Y è la distanza d .

Il risultato è un'onda continua che costituisce una vera e propria **impronta digitale** del contorno dell'oggetto.

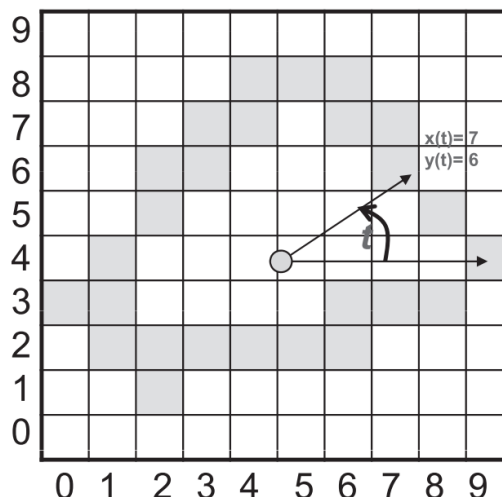


Figura 40: Esempio di "srotolamento" di un contorno in una time-series: asse X l'angolo, asse Y la distanza dal centroide.

Proprietà geometriche:

- **Traslazione: Invariante.** Poiché il sistema di riferimento è ancorato al centroide (che è una proprietà intrinseca della forma) e non al centro dell'immagine, spostare l'oggetto non altera le distanze misurate dal suo baricentro.

- **Scalatura: Sensibile.** Ingrandendo l'immagine, la distanza $d(\theta)$ aumenta, alterando l'ampiezza dell'onda. Si rende invariante normalizzando tutte le distanze (es. dividendo per la distanza massima rilevata).
- **Rotazione: Sensibile.** Ruotando l'oggetto, l'onda trasla orizzontalmente sull'asse X, poiché cambia il punto in cui $\theta = 0$. Si risolve allineando l'angolo 0 all'asse maggiore dell'oggetto, oppure analizzando il segnale nel dominio delle frequenze (Descrittori di Fourier).

3.10 Rappresentazione delle forme

Local Binary Patterns (LBP) È un descrittore di texture molto efficiente. L'idea alla base è descrivere ogni singolo pixel analizzando il contesto locale che lo circonda. Si prende un pixel centrale e si esamina la matrice 3×3 di pixel adiacenti. Il valore di intensità del pixel centrale funge da **soglia**. Per ognuno degli 8 pixel vicini, si confronta il suo valore con quello centrale: se è maggiore o uguale si assegna il bit '1', altrimenti '0'. Leggendo questi 8 bit in senso orario (o antiorario) si ottiene una stringa binaria che viene convertita in un numero decimale (da 0 a 255), il quale andrà a sostituire il pixel centrale originale.

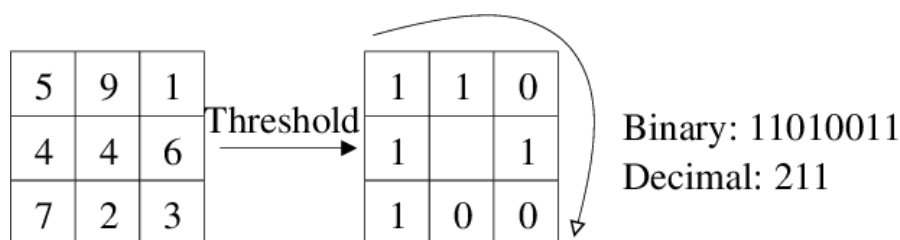


Figura 41: Esempio di calcolo del Local Binary Pattern base su intorno 3×3 .

Questo approccio può essere generalizzato definendo intorni circolari di raggio R con un numero P di pixel campionati, permettendo di analizzare texture a diverse scale.

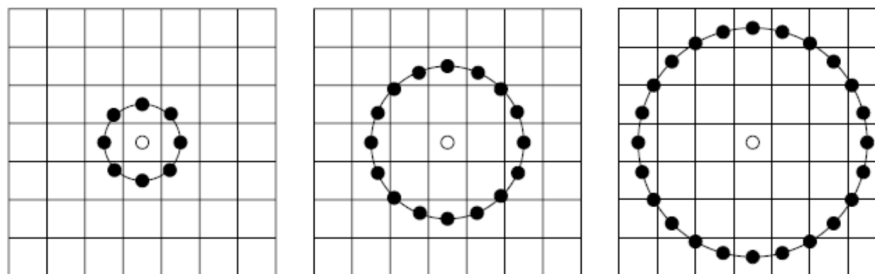


Figura 42: Esempio di LBP esteso ad intorni circolari più grandi.

Histogram of Oriented Gradients (HOG) Il descrittore HOG cattura la distribuzione delle direzioni dei gradienti (ovvero i bordi e le forme locali) all'interno di un'immagine. L'immagine viene suddivisa in piccole porzioni connesse chiamate **celle** (es. griglie di pixel 8×8). Per ogni pixel all'interno della cella si calcola l'ampiezza (magnitudo) e la direzione del gradiente. I gradienti quantizzati vengono poi raggruppati in un istogramma (solitamente a 9 bin direzionali, da 0 a 180 gradi). Ogni pixel "vota" per una specifica direzione, e il peso del suo voto è proporzionale all'ampiezza del suo gradiente. Infine, per garantire robustezza ai cambiamenti di illuminazione, le celle adiacenti vengono raggruppate in **blocchi** più grandi per normalizzare i valori degli istogrammi.

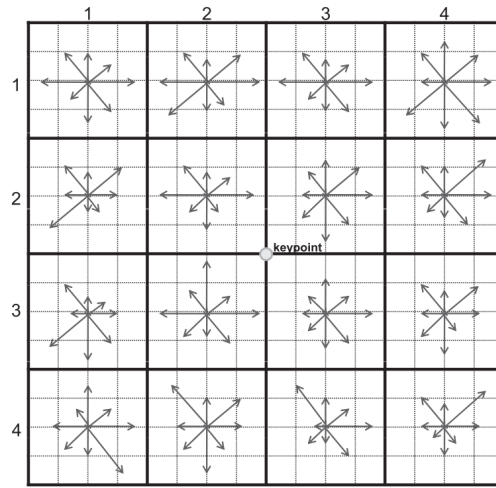


Figura 43: Estrazione delle feature tramite Histogram of Oriented Gradients.

3.10.1 Features locali

Le feature locali consistono nel descrivere un'immagine non nella sua totalità, ma isolando una collezione ristretta di punti "chiave" altamente informativi e distintivi (es. spigoli, angoli o pattern unici).

L'obiettivo principale di un buon estrattore di feature locali (come SIFT o SURF) è individuare punti che siano **robusti**, ovvero identificabili in modo affidabile anche in presenza di:

- Variazioni di illuminazione e contrasto;
- Cambiamenti di scala (zoom);
- Rotazioni dell'immagine;
- Alterazioni del punto di vista (prospettiva affine);
- Presenza di rumore o occlusione parziale.

3.10.2 SIFT (Scale-Invariant Feature Transform)

L'algoritmo SIFT è uno strumento fondamentale per la *computer vision*, poiché permette di estrarre caratteristiche (feature) locali da un'immagine, in modo da poter riconoscere oggetti o parti comuni in fotografie diverse, indipendentemente da scala, rotazione o illuminazione. A differenza delle feature globali (come l'istogramma dei colori), SIFT si concentra su punti di interesse specifici e altamente riconoscibili, detti *keypoint*. Ad esempio, se abbiamo due foto della stessa casa scattate da distanze diverse, i colori globali potrebbero variare molto, ma un *keypoint* corrispondente alla punta del tetto rimarrà riconoscibile.

L'algoritmo si struttura in quattro passi principali:

1. **Scale-space extrema detection:** ricerca dei punti di interesse lavorando a diverse scale di sfocatura.
2. **Keypoint localization:** filtraggio dei punti individuati per mantenere solo i più robusti, eliminando il rumore.
3. **Orientation assignment:** normalizzazione dell'orientamento per garantire l'invarianza alla rotazione.
4. **Keypoint descriptor computation:** creazione dei vettori descrittivi per l'identificazione univoca dei *keypoint*.

1. Creazione dello spazio scalato (Scale-space extrema detection) Il nostro primo obiettivo è riconoscere le stesse forme a prescindere dalla loro grandezza originaria. Per farlo, l'immagine originale viene progressivamente sfocata (operazione detta *blur* o *smoothing*) utilizzando un filtro Gaussiano.

Definiamo il filtro Gaussiano come:

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{\sigma^2}}$$

Come funziona in pratica? Immagina questa funzione come una "campana" tridimensionale centrata sul pixel in esame. Il filtro calcola una media pesata dei pixel circostanti: i pixel più vicini al centro hanno un peso molto alto, mentre quelli più distanti (o ai margini della campana) pesano via via di meno. Il parametro σ (sigma) determina la larghezza di questa campana: aumentandolo, la sfocatura diventa più intensa. L'applicazione di questo filtro avviene tramite convoluzione su tutti i pixel:

$$L(x, y, \sigma) = G(x, y, \sigma) \otimes I(x, y)$$

Questo processo genera una lista di immagini sempre più sfocate (a passi $k\sigma$). Per trovare i potenziali punti salienti, si calcola la differenza tra coppie di sfocature adiacenti (metodo noto come *Difference of Gaussians* o DoG):

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$$

Analizzando queste immagini "differenza", ogni pixel viene confrontato con i suoi 26 vicini: 8 pixel adiacenti nella stessa scala, 9 nella scala superiore e 9 nella scala inferiore. Se il pixel risulta essere un massimo o un minimo locale assoluto rispetto a tutti i 26 vicini, denota un forte cambio di gradiente e diventa un potenziale *keypoint*.

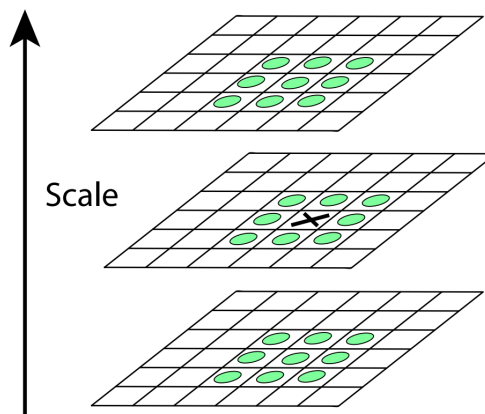


Figura 44: Confronto con i 26 vicini per identificare i potenziali keypoint

2. Localizzazione dei punti chiave (Keypoint localization) In questa fase si "pulisce" l'insieme dei candidati, scartando quelli poco informativi. Vengono eliminati due tipi di punti:

- **Punti a basso contrasto:** considerati come semplice rumore dell'immagine.
- **Punti mal localizzati (es. sui bordi):** un punto situato nel mezzo di una linea retta o di un bordo uniforme non porta informazioni distintive, poiché tutti i punti vicini avranno una variazione di gradiente identica e allineata. Al contrario, si conservano i punti vicini a spigoli o vertici (come gli estremi di una retta), dove il gradiente cambia in più direzioni e il punto diventa univocamente rintracciabile.

3. Assegnazione dell'orientamento (Orientation assignment) Per far sì che l'algoritmo riconosca la nostra "casetta" anche se la foto è capovolta, dobbiamo rendere i keypoint invarianti alla rotazione. Per ogni keypoint isolato, si calcola un istogramma delle direzioni dei gradienti nella regione adiacente, utilizzando la scala σ in cui il punto è stato rilevato. Anche qui si applica una finestra Gaussiana (con parametro 1.5σ) per far sì che i gradienti dei pixel più vicini al keypoint abbiano un "peso" maggiore nel calcolo.

La direzione che rappresenta il picco massimo dell'istogramma diventa l'orientamento ufficiale di quel keypoint. Se ci sono più picchi rilevanti, il keypoint viene duplicato: si mantengono le stesse coordinate, ma si assegnano le diverse orientazioni trovate, per non perdere nessuna potenziale corrispondenza.

4. Calcolo del descrittore (Keypoint Descriptor Computation) L'ultimo passaggio serve a trasformare le informazioni visive in dati numerici (array) facilmente confrontabili dal computer. Si seleziona una regione quadrata di pixel attorno al keypoint (tipicamente 16×16) e la si allinea seguendo l'orientamento assegnato al passo 3, "raddrizzando" di fatto l'area analizzata.

Quest'area viene suddivisa in sotto-griglie (generalmente 4×4). Per ciascuna di queste piccole zone, si raggruppano i vettori dei gradienti calcolando un piccolo istogramma delle direzioni. L'insieme di tutti questi istogrammi forma il **descrittore del keypoint**. Ad esempio, partendo da un'area 8×8 , potremmo ottenere un descrittore riassunto in una matrice 2×2 di gradienti. Grazie a questo array, se in un'altra immagine si rileva uno spigolo simile, il computer confronterà i due descrittori matematicamente per confermare che si tratta dello stesso punto.

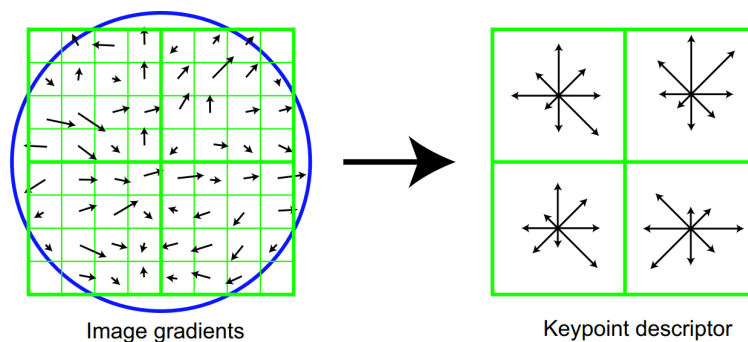


Figura 45: Generazione del descrittore per un keypoint